BANGALORE UNIVERSITY

Jnana Bharathi Campus, Bangalore-560056



DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

A PROJECT REPORT ON

"uniRAG – OFFLINE STUDY ASSISTANT FOR UNIVERSITY STUDENTS"

Submitted by

RAGHAVENDRA CHANDRAKANTHA BHAT
(P03NK24S126033)

Under the guidance of

Dr. Muralidhara B.L
DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

TOWARDS

MAIN PROJECT

Prescribed by the BANGALORE UNIVERSITY
for the IV SEMESTER
MASTER OF COMPUTER APPLICATIONS
For the academic year 2025-2026

BANGALORE UNIVERSITY

Jnana Bharathi Campus, Bangalore-560056

DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

A PROJECT REPORT ON

“uniRAG – OFFLINE STUDY ASSISTANT FOR UNIVERSITY STUDENTS”

CERTIFICATE

This is to certify that **Raghavendra Chandrakantha Bhat (P03NK24S126033)** has satisfactorily completed the project titled **“uniRAG – Offline Study Assistant for University Students”** towards MAIN PROJECT LAB prescribed by the Bangalore University for the 4th Semester Master of Computer Application course in academic year 2025-2026.

Dr. Muralidhara B.L
Senior professor and Guide

Dr. Somashekara M T
Professor and Coordinator

Examiners:

1)

2)

DECLARATION

I hereby declare that the project report, titled "**uniRAG – Offline Study Assistant for University Students**", is prepared in partial fulfillment of the requirements for the award of the degree of Master of Computer Application, Bangalore University for the academic year 2025-2026. Further, the matter embodied in dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

DATE:

PLACE:

RAGHAVENDRA CHANDRAKANTHA BHAT

P03NK24S126033

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Dr. Muralidhara B.L.**, Department of Computer Science and Applications, Bangalore University, for his valuable guidance, constant encouragement, and technical expertise throughout the development of my project, **“uniRAG – Offline Study Assistant for University Students”**. His deep insights into technical skills and his mentorship were instrumental in the successful completion of this report.

I am also thankful to **Dr. Somashekara M T**, Department Coordinator, for his support and for the smooth conduct of the project reviews, and to the Department of Computer Science and Applications, Bangalore University, for providing the necessary resources and an academic environment that fostered this work. The curriculum and the foundational knowledge provided by the department served as the essential framework required to tackle the complexities of building a locally hosted, retrieval-augmented study assistant.

I also thank my classmates, whose use of the application and whose feedback on its layout and answers helped me improve it, and my family for their constant support.

RAGHAVENDRA CHANDRAKANTHA BHAT

ABSTRACT

uniRAG is an offline study assistant for students of the Master of Computer Applications (MCA) programme of Bangalore University. It answers a student's question only from the university's own notes, syllabus and past question papers, and it runs entirely on one laptop, so no data leaves the machine.

The motivation is a common problem in exam preparation. Students work from scattered notes and scanned papers and cannot easily find out how often a topic was asked, in which years or for how many marks. General-purpose chatbots do not solve this: they need the internet, send questions to external servers, may answer from outside the syllabus, and can state wrong facts or invented counts with confidence.

The system is implemented with the Retrieval-Augmented Generation (RAG) approach in Python with a Streamlit interface. Ollama hosts Mistral 7B for questions and explanations, and nomic-embed-text for embeddings. Notes are split into chunks and stored in per-subject Facebook AI Similarity Search (FAISS) indexes, and scanned pages are read with Tesseract OCR. Past papers are converted into a question bank of 1,426 questions from 18 subjects and seven exam years. The central rule is that the language model writes while code counts and checks: every count, year, mark value and trend is computed by deterministic code, and guards stop the model from answering when the notes do not support it. Tests showed that a calibrated matching cut-off of 0.75 gives the best balance (F1 score 0.757), and answers take about 25 seconds for exam facts and one to three and a half minutes for explanations on a CPU-only laptop.

The future scope includes GPU support for faster answers, verification of each sentence against the notes, extraction of figures and diagrams, Kannada and Hindi support, secure campus deployment with e-mail verification, and practice tests.

Table of Contents

ACKNOWLEDGEMENT
ABSTRACT
1. INTRODUCTION
    1.1 INTRODUCTION....1
    1.2 BACKGROUND OF THE PROJECT....1
    1.3 OBJECTIVES....2
    1.4 SCOPE....2
2. EXISTING SYSTEM VS PROPOSED SYSTEM
    2.1 EXISTING SYSTEM....3
    2.2 PROPOSED SYSTEM....3
3. REQUIREMENT ANALYSIS
    3.1 FUNCTIONAL REQUIREMENTS....5
    3.2 NON-FUNCTIONAL REQUIREMENTS....5
    3.3 HARDWARE REQUIREMENTS....6
    3.4 SOFTWARE REQUIREMENTS....6
    3.5 USER REQUIREMENTS....6
4. SYSTEM DESIGN
    4.1 SYSTEM ARCHITECTURE....7
    4.2 MODULE DESCRIPTION....8
    4.3 USE CASE DIAGRAM....9
    4.4 DATA FLOW DIAGRAM....9
5. DATA STORAGE DESIGN AND IMPLEMENTATION
    5.1 DATA STORAGE INTRODUCTION....11
    5.2 DATA STORES USED....11
6. IMPLEMENTATION
    6.1 PYTHON....12
    6.2 STREAMLIT....12
    6.3 OLLAMA....12
    6.4 MISTRAL 7B....12
    6.5 NOMIC-EMBED-TEXT....12
    6.6 LANGCHAIN....13
    6.7 FAISS....13
    6.8 PYPDF AND PYMUPDF....13
    6.9 TESSERACT OCR....13
    6.10 PANDAS AND ALTAIR....13
    6.11 JSON, HASHLIB, HMAC AND SECRETS....13
    6.12 DIFFLIB....14
    6.13 FORMULAE USED....14
7. CODING
    CONFIG.PY....16
    APP.PY – AUTHENTICATION AND NAVIGATION....17

QUERY_UNDERSTANDING.PY – STRUCTURED QUERY.....19
QUESTION_INDEX.PY.....19
QUESTION_MATCHER.PY.....22
EXAM_INTELLIGENCE.PY.....27
RAG_PIPELINE.PY – TREND, FREQUENCY AND ANSWERING.....32
8. TESTING
8.1 INTRODUCTION.....35
8.2 OBJECTIVES OF TESTING.....35
8.3 TYPES OF TESTING.....35
8.4 TEST ENVIRONMENT.....36
8.5 TESTING PROCEDURE.....36
8.6 TEST CASES.....36
8.7 CONCLUSION.....37
9. RESULTS AND DISCUSSION
9.1 EXPERIMENTAL SETUP.....38
9.2 QUESTION BANK.....38
9.3 CALIBRATION OF THE QUESTION-MATCHING CUT-OFF.....40
9.4 REGRESSION SUITE AND RESPONSE TIME.....41
9.5 REPEATED QUESTIONS AND SOLUTIONS BUNDLE.....42
9.6 WORKED EXAMPLES.....43
9.7 DISCUSSION.....43
10. SCREENSHOTS
11. CONCLUSION
12.1 LIMITATIONS.....52
12. FUTURE ENHANCEMENTS
12.1 CALIBRATED MATCHING.....53
12.2 ANSWER VERIFICATION.....53
12.3 GPU AND FASTER INFERENCE.....53
12.4 DIAGRAM AND TABLE EXTRACTION.....53
12.5 MULTILINGUAL SUPPORT.....53
12.6 ACCOUNT SYSTEM AND NETWORK DEPLOYMENT.....53
12.7 REVISION AND PRACTICE TOOLS.....54
12.8 ADDITIONAL PROGRAMMES AND UNIVERSITIES.....54
12.9 BETTER ANALYTICS.....54
13. REFERENCES

List of Tables

1. Table 2.2.1 Existing methods compared with uniRAG.....4
2. Table 3.3.1 Hardware requirements.....6
3. Table 3.4.1 Software requirements.....6
4. Table 4.2.1 Modules and their source files.....8
5. Table 5.2.1 Data stores used in uniRAG.....11
6. Table 8.6.1 List of test cases performed.....37
7. Table 9.2.1 Questions in the question bank per exam year.....38
8. Table 9.2.2 Questions per subject and semester.....39
9. Table 9.2.3 Questions by marks allotted.....40
10. Table 9.3.1 Matching quality at selected cut-offs.....41
11. Table 9.4.1 Response time by type of question (seconds).....42

List of Figures

1. Figure 4.1.1 System architecture of uniRAG.....7
2. Figure 4.1.2 Answer pipeline.....7
3. Figure 4.3.1 Use case diagram.....9
4. Figure 4.4.1 Data flow diagram – Level 0.....10
5. Figure 9.2.1 Questions per exam year.....38
6. Figure 9.2.2 Questions per exam year in each semester.....39
7. Figure 9.2.3 Questions per subject.....39
8. Figure 9.2.4 Share of questions by marks allotted.....40
9. Figure 9.3.1 Precision, recall and F1 score against the cut-off.....41
10. Figure 9.4.1 Median response time by type of question.....42
11. Figure 9.5.1 Source of the 125 repeated-question answers.....43
12. Figure 10.1 Login Page.....45
13. Figure 10.2 Account creation page.....45
14. Figure 10.3 Study Assistant home and subject selection.....46
15. Figure 10.4 Structured answer.....46
16. Figure 10.5 Exam history of a topic.....47
17. Figure 10.6 Question Paper Solver.....47
18. Figure 10.7 Exam Intelligence overview.....48
19. Figure 10.8 Topic year-by-year trend.....48
20. Figure 10.9 Most repeated questions and solutions bundle.....49
21. Figure 10.10 High-weightage questions by marks.....49
22. Figure 10.11 Resource Library.....50
23. Figure 10.12 System Setup.....50
24. Figure 10.13 Upload a document and rebuild the index.....51

1. Introduction

1.1 Introduction

India has one of the largest higher-education systems in the world. The All India Survey on Higher Education (AISHE) 2021-22 of the Ministry of Education recorded about 4.33 crore (43.3 million) students enrolled in higher education, up from 4.14 crore in 2020-21, an increase of about 19 lakh in a single year. Each of these students prepares for university examinations, and for most of them the preparation material is a pile of notes and past papers. The papers of this project show how large that pile is: the first three semesters of the Bangalore University MCA programme alone contain 1,426 examination questions from 18 subjects over seven exam years (2020 to 2026), and 125 of these questions were repeated in two or more sittings. A student who wants to know which questions repeat has to read through all of them.

Preparing for a university examination is, to a large extent, a search problem. A student has the syllabus, several thick reference books or lecture notes, and a pile of previous years' question papers, often as scanned images. The questions the student really needs answered are practical ones: What does this topic mean? How many times has it been asked? In which years? For how many marks? Is it worth reading at all? Answering them by hand means leafing through dozens of pages, and the answers are rarely complete.

uniRAG is an offline study assistant built for the Master of Computer Applications (MCA) programme of Bangalore University. It answers a student's question only from the university's own notes, syllabus and past question papers, and it runs entirely on one laptop: no internet connection, no cloud service and no external API is used, so no student data leaves the machine. Each university can run its own separate installation with its own documents.

The system follows the Retrieval-Augmented Generation (RAG) approach. Instead of asking a language model to answer from its general memory, uniRAG first retrieves the relevant passages from the stored documents and then asks a locally hosted language model to write an explanation using only that material. Everything that can be computed exactly, such as how often a topic appeared, the years, the marks and the trend, is computed by ordinary program code from a structured question bank and is never left to the model.

1.2 Background of the Project

Large language models can explain almost any topic, but using them directly for exam preparation has well-known problems. Cloud chatbots need an internet connection and a subscription, and they transmit every question to an external server. They have no knowledge of what a particular university actually teaches or asks, so they can answer from outside the syllabus or from an unrelated textbook. They also “hallucinate”: they can state a wrong fact, or invent a number such as “this topic has been asked five times”, with complete confidence.

At the same time, small language models that can run on an ordinary laptop have become good enough to write readable explanations when they are given the right context. Tools such

as Ollama make it possible to host such models locally, and libraries such as Facebook AI Similarity Search (FAISS) make it possible to search thousands of text chunks in milliseconds on a CPU. These developments made it practical to build a study assistant that is private, free to run and grounded in the university's own material.

The project therefore started from a simple observation: the real value for an MCA student lies in combining two things, explanations drawn from the student's own notes and facts drawn from the past question papers. The current version indexes 22 subject collections (theory and laboratory) with 96,367 text chunks of notes, and a question bank of 1,426 questions extracted from question papers of 18 subjects between 2020 and 2026.

1.3 Objectives

The main objectives of the project are:

- To build a study assistant that works completely offline and keeps all student data on the local machine.
- To answer questions from the university's own notes, syllabus and past papers, and to decline politely when a topic is outside them.
- To turn scanned question papers into a clean, searchable question bank by combining text extraction and OCR.
- To compute exam statistics (how many times a topic was asked, in which years, for how many marks, its yearly trend and the most repeated questions) with deterministic code so that numbers are never invented by the model.
- To provide a Question Paper Solver that extracts every question from an uploaded paper and answers each one with a downloadable answers file.
- To provide role-based access, with students who register themselves and an administrator who manages documents and rebuilds the indexes.
- To run acceptably on a CPU-only laptop with 16 GB of RAM, without a GPU.

1.4 Scope

uniRAG is designed for a single university programme at a time. The present installation covers the first three semesters of the MCA programme of Bangalore University (18 subjects with question papers, plus laboratory subjects). The scope of the system includes:

- Explaining a topic in a textbook-style explanation followed by a plain-language analogy, with the source files listed.
- Answering questions about the examination history of a topic: frequency, exact past questions by year and marks, trend and study guidance.
- Writing laboratory programs following the approach of the lab manual (C by default, Java for the object-oriented lab or when Java is asked for).
- Question Paper Solver, Exam Intelligence (topic trend, most repeated questions with a solutions PDF, high-weightage questions), Resource Library and System Setup.
- Reading scanned PDF pages with Tesseract OCR.

2. Existing System vs Proposed System

2.1 Existing System

Students currently prepare through a combination of the following methods:

- **Textbooks and PDF notes.** Material is spread across many files. Finding what a book says about one topic needs manual searching or a plain keyword search that does not understand spelling mistakes, abbreviations or related wording.
- **Previous year question papers.** Papers are available as scanned images, with several years often merged into one file. To find how often a topic was asked, a student must read every paper by hand and count. The result is slow, easily wrong, and is rarely shared.
- **Seniors, friends and messaging groups.** Advice about “important topics” is based on memory and opinion, not on data.
- **General chatbots and online services.** They need internet access, send questions to external servers, are not aware of the university's syllabus and papers, and can produce wrong or invented statements, including invented frequency counts.

The main limitations of the existing approach are therefore the lack of a single searchable source, the effort needed to analyse papers, the absence of reliable exam statistics, dependence on the internet and loss of privacy, and answers that are not tied to the syllabus.

2.2 Proposed System

The proposed system, uniRAG, brings notes, syllabus and question papers into one locally hosted application. Documents are indexed once by the administrator. After that, a student can ask a question in ordinary language, even with spelling mistakes or mixed-language wording such as “queue kitni baar aaya hai exam me”, and receive a grounded answer together with exam facts computed from the question bank.

The important design decision is the division of work between the model and the code. The language model is used for what it does well, namely understanding a messy question and writing a readable explanation. Code is used for everything that must be exact: counting questions, listing years and marks, classifying a trend, checking that the topic really appears in the retrieved notes and deciding whether a topic is inside the syllabus.

Aspect	Existing methods	uniRAG (proposed)
Internet	Needed for chatbots and online services	Not needed; runs fully offline
Privacy	Questions sent to external servers	All data stays on the local machine
Source of answers	General knowledge or unrelated books	Only the university's notes, syllabus and papers
Out-of-syllabus questions	Answered confidently	Declined with an explanation
Exam statistics	Manual counting or opinion	Counted by code from a question bank of 1,426 questions

Aspect	Existing methods	uniRAG (proposed)
Scanned papers	Read by hand	Read with OCR and converted into structured questions
Messy questions	Keyword search fails	Typos, abbreviations and mixed language are understood
Cost	Subscription or API fee	Free; open-source tools only

Table 2.2.1 Existing methods compared with uniRAG

3. Requirement Analysis

3.1 Functional Requirements

1. **User registration and login.** A student can register with a name, e-mail and password and log in. The administrator logs in with an administrator password.
2. **Role-based access.** Students see the Study Assistant, Question Paper Solver, Exam Intelligence and Resource Library. The administrator additionally sees System Setup.
3. **Subject scope.** The user can select a subject so that retrieval and statistics are restricted to it.
4. **Study questions.** The system shall understand a free-text question and return a structured answer with an explanation, an analogy, exam history, trend and guidance where relevant.
5. **Exam facts.** The system shall report the number of matching past questions, the years, marks and verbatim questions from the question bank.
6. **Out-of-syllabus handling.** The system shall decline a plain definition question about a topic that is in neither the syllabus nor any past paper.
7. **Lab programs.** The system shall write laboratory programs when asked, in C by default and in Java for the object-oriented lab or when the question asks for Java.
8. **Question Paper Solver.** A student can upload a question paper PDF; the system extracts the questions, answers each, and offers the answers as a download.
9. **Exam Intelligence.** Topic trend year by year, most repeated questions with a solutions PDF, and high-weightage questions by marks.
10. **Resource Library.** The student can browse the notes and papers that are indexed.
11. **System Setup (administrator).** Upload documents, set university name and department, and rebuild the indexes and question bank.
12. **OCR.** Pages with no text layer shall be read using Tesseract OCR.

3.2 Non-Functional Requirements

- **Privacy.** No data shall leave the machine; no cloud API is used.
- **Accuracy and trust.** Counts, years and marks shall come from code, not from the model; answers shall list their sources; guards shall prevent answers from unsupported text.
- **Performance.** An answer should be produced in a few minutes on a CPU-only laptop. In the evaluation run, exam-fact answers took about 25 seconds and explanations and lab programs took about one to three and a half minutes.
- **Security.** Passwords shall be stored only as salted hashes; repeated wrong passwords shall be locked out temporarily; one user's chat shall not be visible to the next user.
- **Reliability.** Data files shall be written safely (temporary file then atomic replace); a corrupt data file shall never be overwritten without a backup.
- **Usability.** The interface shall be clear and uncluttered, with a consistent layout and colour scheme.

- **Maintainability.** The system shall be modular, with configuration in a single file, so a new university can be set up without editing the source code.
- **Portability.** The application shall run on Windows, Linux and macOS with Python 3.12.

3.3 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (or equivalent) or better; developed on an Intel Core i5-13420H
RAM	16 GB recommended (a 7B model is loaded in memory)
GPU	Not required; inference runs on the CPU
Storage	About 5 GB for the two models plus space for PDFs and indexes
Machine used	Lenovo IdeaPad Slim 3 laptop
Display	Any display with a modern web browser

Table 3.3.1 Hardware requirements

3.4 Software Requirements

Software	Purpose
Windows 11 (Linux / macOS also possible)	Operating system; the project was developed on Windows 11
Python 3.12	Programming language and runtime
Streamlit	Web user interface
Ollama	Hosts the local language and embedding models
Mistral 7B (mistral:7b)	Question understanding, explanations and lab code
nomic-embed-text	Text embeddings for semantic search
LangChain, langchain-community, langchain-ollama	Text splitting, document objects and Ollama integration
FAISS (faiss-cpu)	Vector similarity search
pypdf, PyMuPDF	Reading PDFs and producing PDF downloads
Tesseract OCR, pytesseract, Pillow	OCR of scanned pages
Web browser	Chrome, Edge or Firefox to open the Streamlit page

Table 3.4.1 Software requirements

3.5 User Requirements

Student: A student needs to ask questions in natural language, see clear answers with sources, find out how often and when a topic was asked, solve a whole question paper, and download answers. The student does not need technical knowledge; the only requirement is a browser.

Administrator: The admin/faculty/project owner needs to add notes, syllabus and question papers, set the university name, and rebuild the indexes once after adding documents. The administrator must set a private administrator password before sharing the application.

4. System Design

4.1 System Architecture

uniRAG is built in four layers. The presentation layer is the Streamlit application, which handles login, tabs and display. The application layer is a set of Python modules for query understanding, retrieval, question matching, exam statistics and document loading. The local AI services are two Ollama models. The data layer consists of local files: the PDF library, per-subject Facebook AI Similarity Search (FAISS) indexes, the question bank and the student file. Nothing in the system communicates with the internet at run time.

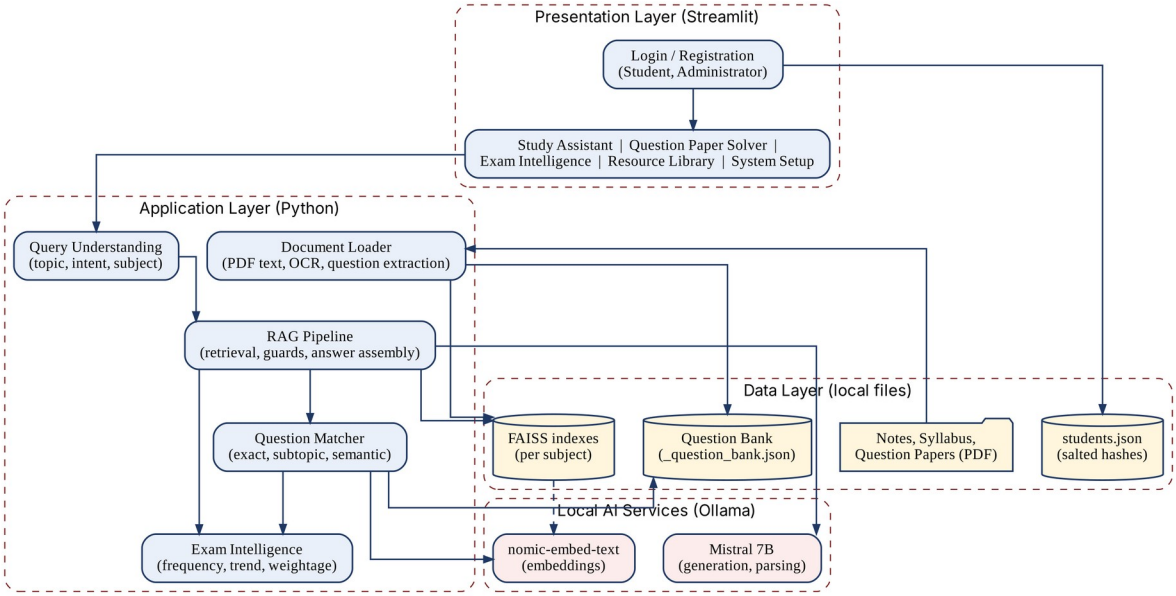


Figure 4.1.1 System architecture of uniRAG

When a student asks a question, the pipeline shown in Figure 4.1.2 is followed. The question is first parsed into a structured query. Retrieval then collects evidence from the notes and from the question bank. Guards decide whether the evidence is good enough. Only then is the language model asked to write the explanation, and the page assembles the final answer from the model's text and the facts computed by code.

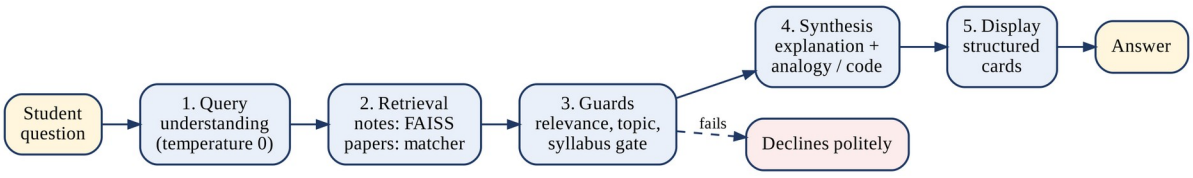


Figure 4.1.2 Answer pipeline

4.2 Module Description

The project is organised into the following modules. Table 4.2.1 lists the source file of each.

Module	File	Responsibility
User interface	app.py	Login, registration, tabs, chat, Question Paper Solver, Exam Intelligence, Resource Library, System Setup, PDF export
Configuration	config.py	Paths, model names, chunk sizes, token limits, university name and administrator password
Document loader	document_loader.py	PDF loading, OCR fallback, year detection and question extraction
Query understanding	query_understanding.py	Turns a messy question into a structured query (topic, intents, subject, scope)
RAG pipeline	rag_pipeline.py	Index building, retrieval, guards, trend and frequency analysis, answer assembly
Question index	question_index.py	Question bank records and the question vector store
Question matcher	question_matcher.py	Exact, subtopic and semantic matching of questions
Exam intelligence	exam_intelligence.py	Syllabus check, study guidance, most repeated topics, marks analysis
Curriculum	curriculum.py	Answers programme questions (for example the subjects of a semester) from folder names
Repeated solutions	repeated_solutions.py	Stores and exports model answers for the most repeated questions
Evaluation	eval_questions.py, test_uniRAG.py	Regression suite and automated unit tests

Table 4.2.1 Modules and their source files

Study Assistant

The student types a question. Query understanding extracts the topic and the intents (definition, frequency, exact questions, trend, ranking, lab program and so on). Notes are searched in the FAISS index of the selected subject. For exam-related intents the question matcher searches the question bank. The reply is divided into sections (concept, exam history, trend, guidance, sources), each shown in its own card.

Question Paper Solver

The student uploads a paper. The document loader extracts its text, using OCR for scanned pages, the extractor splits the text into individual questions, and each question is answered through the same pipeline in a plain-answer mode that omits exam commentary. All answers can be downloaded.

Exam Intelligence

This module has three views. The Topic Year-by-Year Trend counts the questions that literally match a topic in each exam year (zero-filled for years without a match) and classifies the trend as rising, falling or stable. Most Repeated Questions lists questions asked in two or

more exam sittings of a subject and bundles a model answer for each into one PDF. High-Weightage Questions by Marks lets the student choose a marks level (for example 10 marks) and lists the past questions that carried it.

Resource Library and System Setup

The Resource Library lists the documents that are indexed. System Setup is visible only to the administrator; it uploads documents, stores the university name and department, and offers a single “Rebuild everything now” action that OCRs scanned pages, embeds the notes and rebuilds the question bank.

4.3 Use Case Diagram

There are two actors. The Student registers, logs in and uses the four study features. The Administrator logs in with the administrator password and manages documents and indexes. Figure 4.3.1 shows the use cases.

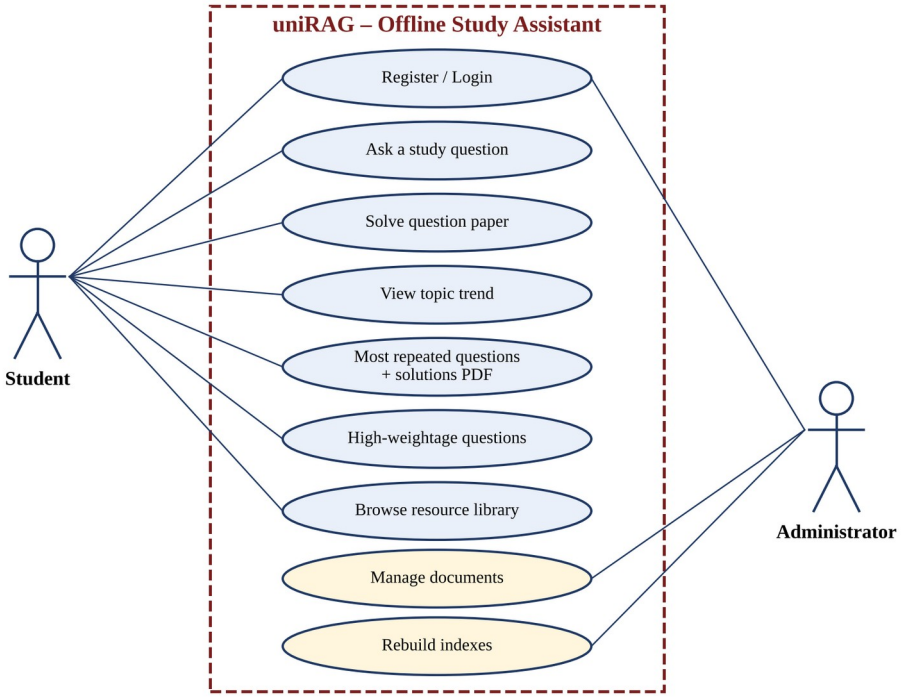


Figure 4.3.1 Use case diagram

4.4 Data Flow Diagram

The context-level diagram (Level 0) shows uniRAG as a single process exchanging data with the two external entities, the student and the administrator.

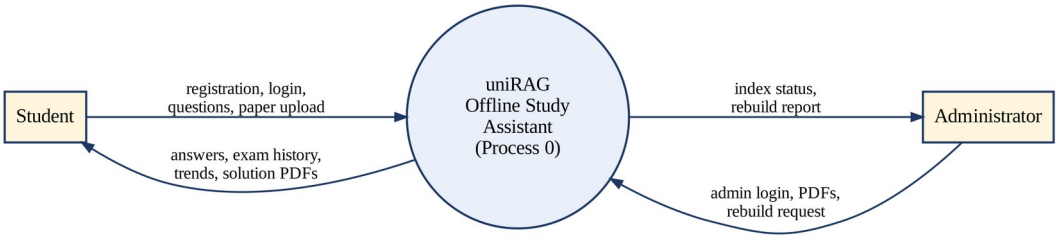


Figure 4.4.1 Data flow diagram – Level 0

Figure 4.4.1 is read from left to right. The student on the left is the main user of the system. A student sends four kinds of data to it: registration details, login credentials, study questions and uploaded question papers. In return the system sends back whatever the student asked for, which is an answer with its sources, the exam history of a topic, a year-wise trend, the solutions PDF of the repeated questions, or the answers of an uploaded paper.

The administrator on the right is the second external entity. The administrator logs in with the administrator password, supplies the learning material in the form of notes, syllabus and question-paper PDFs, and asks the system to rebuild its indexes. The system replies with the status of the indexes and a rebuild report, so that the administrator knows whether the documents were processed correctly.

At this level the inside of the system is deliberately not shown. Retrieval, question matching, the guards and answer generation are all hidden inside the single circle (Process 0), and the diagram shows only what enters and what leaves it. Notably, no data flows to any outside server or cloud service, because no such entity exists in the diagram: the whole exchange happens between the two users and the one local application.

5. Data Storage Design and Implementation

5.1 Data Storage Introduction

Every information system needs a reliable way to store its data. In a typical web application this is a relational database. uniRAG has a different workload: its main data is a library of documents that must be searched by meaning, a question bank of about 1,400 records that is written in bulk by a rebuild and then only read, and a small file of student accounts. The system runs for one user at a time on one machine. For this workload a combination of JSON files and FAISS vector indexes is simpler, needs no database server, can be copied with the project folder and can be rebuilt from the source PDFs at any time.

5.2 Data Stores Used

Two kinds of stores are used. A JSON file holds structured records and is read and written with Python's standard `json` module. A FAISS index holds embedding vectors together with the text chunk and metadata of each vector, and is searched by similarity using embeddings produced by nomic-embed-text.

Store	Format	Purpose
students.json	JSON	Student accounts: name, salt and password hash, keyed by e-mail
_question_bank.json	JSON	Structured questions extracted from past papers (1,426 records)
_manifest.json	JSON	Build information: version, time, status and chunk counts per subject
_answer_cache.json	JSON	Saved answers, reused when the model input is identical
solutions_index.json, solutions_generated.json	JSON	Curated and model-generated answers for repeated questions
university_settings.json	JSON	University name and department set from the Setup tab
faiss_store/<subject>	FAISS	Per-subject vector index of the notes chunks
faiss_store/_question_bank_faiss	FAISS	Vector index of individual questions for semantic matching

Table 5.2.1 Data stores used in uniRAG

6. Implementation

6.1 Python

Python is a high-level, interpreted, general-purpose programming language known for its readable syntax and its very large ecosystem of libraries. It supports object-oriented, procedural and functional styles. uniRAG is written entirely in Python 3.12: the user interface, document processing, retrieval, statistics and the glue to the language models. Python's standard library also provides the JSON, hashing and file-handling facilities that the data storage of the project relies on.

6.2 Streamlit

Streamlit is an open-source Python framework for building interactive web applications without writing HTML, CSS or JavaScript by hand. A script is re-run from top to bottom whenever the user interacts with a widget, and widgets such as tabs, buttons, file uploaders, chat inputs and charts are created with simple function calls. State that must survive a re-run is kept in `st.session_state`. uniRAG uses Streamlit for the whole interface: login, tabs, chat, uploads, charts and downloads. The visual design (colours, cards, sidebar) is applied with a custom CSS block.

6.3 Ollama

Ollama is a platform that downloads, manages and runs large language models on the user's own computer and exposes them through a local interface. Because the model runs locally, no text leaves the machine and no API fee is charged. The speed depends on the CPU, RAM and any GPU. In uniRAG, Ollama hosts both the generation model and the embedding model, and keeps the model loaded between questions (`keep_alive` of 30 minutes) to avoid the 10–30 second cost of reloading a 7B model.

6.4 Mistral 7B

Mistral 7B is an open-weight language model with about seven billion parameters. It is the largest model that runs acceptably on the development laptop. Smaller models (llama3.2:3b and qwen2.5:3b) were tried; they were faster but noticeably worse at writing code, so Mistral 7B was kept. It is used to parse a question into a structured query (at temperature 0, so the same question always parses the same way) and to write explanations, analogies and laboratory code (at temperature 0.1). The number of tokens it may generate is capped according to the marks of the question, which bounds the worst-case response time on a CPU.

6.5 nomic-embed-text

nomic-embed-text is a text embedding model that converts a piece of text into a 768-dimensional vector so that texts with similar meaning have vectors that lie close together. uniRAG uses it to embed the chunks of the notes and the individual questions of the question bank, and to embed a student's query at search time.

6.6 LangChain

LangChain is a library of building blocks for applications that use language models. uniRAG uses only a few of them: the `RecursiveCharacterTextSplitter` to cut documents into chunks of 400 characters with a 50-character overlap, the `Document` class to carry text and metadata, the `FAISS` vector-store wrapper, and the `ChatOllama` and `OllamaEmbeddings` classes that connect to Ollama.

6.7 FAISS

FAISS (Facebook AI Similarity Search) is a library for fast similarity search over dense vectors. It runs well on a CPU. uniRAG builds one FAISS index per subject, because a single combined index mixed subjects (a small subject's notes were crowded out by a much larger one) and was slower to search. A merged index is kept only as a fallback for the “All / Auto-detect” option. A separate index holds the embeddings of the individual questions.

6.8 PyPDF and PyMuPDF

PyPDF (through the LangChain loader) extracts the text layer of PDF pages. PyMuPDF renders pages to images for OCR and is used to produce the PDF downloads, such as the solutions bundle for the most repeated questions. Pages whose extracted text is too short are treated as scans and sent to OCR.

6.9 Tesseract OCR

Tesseract is an open-source optical character recognition engine. It is installed separately from Python and is called through the `pytesseract` wrapper, with `Pillow` handling the page images. It is used for scanned question papers (which are read page by page, with the exam year taken from the printed header and carried forward over continuation pages) and for the scanned syllabus.

6.10 Pandas and Altair

Streamlit uses pandas data frames and the Altair charting library for tables and charts. uniRAG uses them to show the year-by-year trend of a topic as a bar chart and the ranking tables of the Exam Intelligence tab.

6.11 JSON, hashlib, hmac and secrets

JSON is the file format for all structured records. The standard-library modules `hashlib`, `hmac` and `secrets` implement password security: a random salt is generated with `secrets`, the password is hashed with PBKDF2-HMAC-SHA256 (100,000 iterations) using `hashlib`, and the stored and computed hashes are compared in constant time with `hmac.compare_digest`. No extra dependency is needed.

6.12 difflib

The standard `difflib` module provides fuzzy string matching. The question matcher uses it for near-exact matches, and the pipeline uses it to correct the spelling of a topic against the words that occur in the question papers.

6.13 Formulae Used

uniRAG uses a small number of mathematical formulae. They are listed here with the place where each one is applied. The numbers in brackets are used to refer to them.

Distance between embeddings (FAISS)

Every chunk of the notes and the query itself is converted by nomic-embed-text into a vector of 768 numbers. The FAISS flat index compares the query vector q with a chunk vector c by the squared Euclidean (L2) distance. A smaller value means that the texts are closer in meaning.

$$d(q, c) = \Sigma\ (qi - ci)^2,\ \ i = 1 \ldots 768 \quad (6.1)$$

A chunk is used as evidence only if its distance is at most 1.15 (`MAX_RELEVANT_DISTANCE`). For matching a question against the question bank, the stricter cut-off 0.75 chosen by calibration is used (Chapter 9).

Number of chunks

A document of L characters is cut into chunks of size S = 400 characters with an overlap V = 50 characters, so every new chunk advances by S − V = 350 characters. The approximate number of chunks is

$$N \approx \lceil\ (L - V) / (S - V)\ \rceil \quad (6.2)$$

Fuzzy string similarity (difflib)

The question matcher and the spelling correction compare two strings A and B with the ratio of Python's `difflib.SequenceMatcher`, where M is the number of matching characters and T is the total number of characters in both strings:

$$ratio(A, B) = 2M / T \quad (6.3)$$

A ratio of at least 0.75 counts as a near-exact match of a topic with a question, a ratio above 0.8 is accepted for a subtopic, and a word is corrected to a known word of the papers when its ratio is at least 0.82.

Trend classification

For a topic, the years with a match are sorted in order and their yearly counts v1 … vn are split into an earlier half and a later half. With the averages of the two halves, the trend is decided by a margin of 0.5 question:

rising if avg(later) > avg(earlier) + 0.5.....
falling if avg(later) < avg(earlier) − 0.5.....
stable otherwise.....(6.4)

Coverage of a topic

The Exam Intelligence answer states in how many sittings a topic was asked. With D the number of distinct exam years in which the topic appeared and Y the number of sittings on record for the subject,

Coverage (%) = 100 × D / Y.....(6.5)

For example, the topic Queue in Data Structures appeared in 5 of the 6 sittings on record, a coverage of 100 × 5 / 6 ≈ 83 per cent.

Evaluation of the matching cut-off

To calibrate the cut-off, the questions returned for each test query were compared with the answer key. TP is the number of correct questions returned, FP the number of wrong questions returned and FN the number of correct questions missed:

Precision = TP / (TP + FP).....(6.6)

Recall = TP / (TP + FN).....(6.7)

F1 = 2 × Precision × Recall / (Precision + Recall).....(6.8)

At the chosen cut-off 0.75 these gave a precision of 0.777, a recall of 0.737 and an F1 score of 0.757.

Password hashing and answer-cache key

A password is never stored. The stored value is the PBKDF2 derived key, computed with HMAC-SHA256 as the pseudo-random function and 100,000 iterations over a random salt. The saved answers are looked up by a SHA-256 digest of everything that influences the answer:

key = SHA-256 (model, prompt, retrieved notes, question, marks guidance, token limit)
(6.9)

stored hash = PBKDF2-HMAC-SHA256 (password, salt, 100000).....(6.10)

7. Coding

This chapter lists the main source code of uniRAG. The complete project also contains helper and maintenance scripts that are not reproduced here.

config.py

```
# config.py

from pathlib import Path

# Project paths
BASE_DIR = Path(__file__).parent
DATA_DIR = BASE_DIR / "data" / "bangalore_university_mca"
NOTES_DIR = DATA_DIR / "notes"
PAPERS_DIR = DATA_DIR / "question_papers"
SYLLABUS_DIR = DATA_DIR / "syllabus"
FAISS_DIR = BASE_DIR / "faiss_store"

# Ollama models
LLM_MODEL = "mistral:7b"
EMBEDDING_MODEL = "nomic-embed-text"

# --- LLM speed tuning (CPU-only inference) ---
# num_ctx: how much context window Ollama allocates. Our prompts (context +
# question + guidance) are well under 1500 tokens even at the largest marks
# level, so 2048 gives headroom without over-allocating.
LLM_NUM_CTX = 3072
# num_thread: leave as None to let Ollama auto-detect your CPU core count
# (usually correct) — only set this if you've confirmed a specific value
# helps, since guessing wrong can make things slower, not faster.
LLM_NUM_THREAD = None
# Hard cap on how many tokens the model will generate, per marks level.
# This bounds worst-case latency directly — generation time scales with
# token count on CPU, so a runaway 10-mark answer can't take forever.
MARKS_MAX_TOKENS = {2: 250, 5: 600, 8: 850, 10: 1100}
# Code answers (lab programs / "write an algorithm") need more room than prose.
LAB_MAX_TOKENS = 900
# If the model hits the token cap mid-answer, ask it to finish (at most this many extra calls, each
# limited to CONTINUATION_TOKENS). If it is still cut off, the answer is trimmed to its last
complete
# sentence so it never ends in the middle of a word, sentence or code block.
MAX_CONTINUATIONS = 1
CONTINUATION_TOKENS = 350

# Path to the Tesseract-OCR engine executable. Leave as None if `tesseract`
# is already on your system PATH. On Windows, if you installed it via the
# UB-Mannheim installer and it's not on PATH, set the actual install path
# here, e.g.:
#   TESSERACT_CMD = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
TESSERACT_CMD = r"C:\Program Files\Tesseract-OCR\tesseract.exe"

# RAG settings
CHUNK_SIZE = 400
CHUNK_OVERLAP = 50
TOP_K = 3

# University info — overridable via university_settings.json (written by
# the in-app Setup screen), so a new university can be onboarded without
# editing source code. Falls back to these defaults if that file doesn't
# exist yet or can't be read.
UNIVERSITY_NAME = "Bangalore University"
DEPARTMENT = "MCA"

_SETTINGS_FILE = BASE_DIR / "university_settings.json"
if _SETTINGS_FILE.exists():
    try:
        import json as _json
        with open(_SETTINGS_FILE, "r", encoding="utf-8") as _f:
            _settings = _json.load(_f)
        UNIVERSITY_NAME = _settings.get("university_name", UNIVERSITY_NAME)
        DEPARTMENT = _settings.get("department", DEPARTMENT)
    except Exception:
        pass  # a corrupt settings file should never crash startup — fall back to defaults

# Shared admin password gating document upload/reindexing — NOT per-student
```

```
# accounts. Change this before handing the app to anyone else, or set the
# ADMIN_PASSWORD environment variable instead of editing this file.
import os as _os
ADMIN_PASSWORD = _os.environ.get("ADMIN_PASSWORD", "")
if not ADMIN_PASSWORD or ADMIN_PASSWORD == "bu":
    # SAFETY_FIX: never fall back to a guessable password
    import secrets as _secrets
    ADMIN_PASSWORD = _secrets.token_urlsafe(9)
    print(f"[uniRAG] ADMIN_PASSWORD is not set. Admin password for THIS run only:
{ADMIN_PASSWORD}")
    print('[uniRAG] To choose your own, set it before starting:  $env:ADMIN_PASSWORD="your-
password"')

# HARDENING_PATCH_APPLIED
# True  = the admin "Rebuild everything" button backs up and then RESTORES the existing
#         question bank (so verified/corrected questions are never lost).
# False = old behaviour (re-extract the question bank from the PDFs).
PRESERVE_QUESTION_BANK_ON_REBUILD = True

# Model used ONLY for query parsing (short JSON output). Smaller = faster on CPU.
# Leave as LLM_MODEL for no change. To try the faster parser, run `ollama pull llama3.2:3b`
# and change the next line to: PARSE_MODEL = "llama3.2:3b"  (then run eval_questions.py).
PARSE_MODEL = LLM_MODEL

# Keep models loaded between questions (Ollama unloads after 5 minutes by default,
# and reloading a 7B model costs 10-30 s).
LLM_KEEP_ALIVE = "30m"

# ANSWER_CACHE_PATCH: True = reuse saved answers when the model's input is identical (fast).
# Set to False when running eval_questions.py so every answer is generated fresh.
ANSWER_CACHE = True
```

app.py – authentication and navigation

```
# Passwords are hashed (PBKDF2-HMAC-SHA256, stdlib only, no new dependency)
# — appropriate security for an offline single-laptop deployment, not a
# production system exposed to the internet: no rate limiting, no email
# verification, no password-reset flow.
STUDENTS_FILE = BASE_DIR / "students.json"


def _hash_password(password, salt):
    return hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt.encode('utf-8'),
100_000).hex()


class StudentFileUnreadable(Exception):
    """students.json exists but could not be understood. Never overwrite it in this state."""


def _read_students_file():
    """Returns (data, ok). ok=False means the file exists but could not be read or understood."""
    try:
        raw = STUDENTS_FILE.read_bytes()
    except OSError:
        return {}, False                      # locked by another program, permissions, ...
    for enc in ("utf-8-sig", "utf-16"):       # utf-16 = what PowerShell's ">" writes
        try:
            text = raw.decode(enc)
        except UnicodeDecodeError:
            continue
        if not text.strip():
            return {}, True                   # a blank file is just "no students yet"
        try:
            data = json.loads(text)
        except ValueError:
            continue
        if isinstance(data, dict):
            return data, True
    return {}, False


def _load_students(strict=False):
    if not STUDENTS_FILE.exists():
        return {}
    data, ok = _read_students_file()
    if ok:
        return data
```

```
    # SAFETY_FIX: keep a copy so a later save cannot destroy it
    try:
        import shutil as _sh
        _bk = str(STUDENTS_FILE) + ".corrupt"   # one backup, not one per call
        if not os.path.exists(_bk):
            _sh.copy2(STUDENTS_FILE, _bk)
    except Exception:
        pass
    if strict:
        raise StudentFileUnreadable("students.json could not be read")
    return {}


def _save_students(students):
    _tmp = str(STUDENTS_FILE) + ".tmp"
    with open(_tmp, 'w', encoding='utf-8') as f:
        json.dump(students, f, indent=2)
    os.replace(_tmp, STUDENTS_FILE)


def _register_student(email, name, password):
    students = _load_students(strict=True)   # refuse to save over an unreadable file
    salt = secrets.token_hex(16)
    students[email.lower().strip()] = {
        "name": name.strip(),
        "salt": salt,
        "password_hash": _hash_password(password, salt),
    }
    _save_students(students)


def _verify_student(email, password):
    students = _load_students()
    record = students.get(email.lower().strip())
    if not record:
        return False
    return hmac.compare_digest(_hash_password(password, record["salt"]),
str(record["password_hash"]))


# Login lockout: 5 wrong tries -> 60 s wait. Kept per-process (survives page refresh).
_MAX_FAILS = 5
_LOCK_SECONDS = 60


@st.cache_resource
def _login_attempts():
    return {}


def _login_locked(key):
    """Seconds left in the lockout for this key, or 0 if not locked."""
    rec = _login_attempts().get(key)
    if rec and rec["fails"] >= _MAX_FAILS:
        left = int(rec["last"] + _LOCK_SECONDS - time.time())
        if left > 0:
            return left
        _login_attempts().pop(key, None)
    return 0


def _login_failed(key):
    rec = _login_attempts().setdefault(key, {"fails": 0, "last": 0.0})
    rec["fails"] += 1
    rec["last"] = time.time()


def _login_reset(key):
    _login_attempts().pop(key, None)


# ... (page layout and styling omitted) ...

_is_admin = st.session_state.role == "admin"
_nav_tabs = [
    " Study Assistant",
    " Question Paper Solver",
    " Exam Intelligence",
    " Resource Library",
] + ([" System Setup"] if _is_admin else [])

_tabs = st.tabs(_nav_tabs)
```

```
tab_ask = _tabs[0]
tab_solver = _tabs[1]
tab_intel = _tabs[2]
tab_library = _tabs[3]
tab_admin = _tabs[4] if _is_admin else None
```

query_understanding.py – structured query

```
@dataclass
class StructuredQuery:
    raw_query: str
    canonical_topic: str
    subtopics: List[str] = field(default_factory=list)
    intents: List[str] = field(default_factory=lambda: ["definition"])
    search_queries: List[str] = field(default_factory=list)
    question_scope: str = "related"  # "exact", "subtopic", "related"
    subject: Optional[str] = None
    marks: Optional[int] = None
    year_constraint: Optional[str] = None
    is_followup: bool = False
    resolved_reference: Optional[str] = None
    needs_notes: bool = True
    needs_question_bank: bool = False
    needs_syllabus: bool = False
    needs_lab: bool = False

    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)



def _topic_matches_query(topic: str, query: str) -> bool:
    """False when the assigned canonical_topic shares no real word (and no close
    spelling-variant prefix) with the actual question — e.g. a header reading
    "NORMALIZATION" over an answer about "Data cleaning". True (permissive) whenever
    either side has no extractable words, so this never fires on something it can't
    meaningfully judge."""
    t = set(re.findall(r"[a-z]{3,}", (topic or "").lower()))
    q = set(re.findall(r"[a-z]{3,}", (query or "").lower()))
    if not t or not q:
        return True
    return bool(t & q) or any(a.startswith(b[:5]) or b.startswith(a[:5]) for a in t for b in q)

def _looks_like_followup(question: str) -> bool:
    """A question is a genuine follow-up only if it points back with a pronoun
    ("it", "that"...) OR has no content words of its own ("how many times?",
    "show exact questions"). A new question that merely lacks a topic name
    ("how many subjects in 1st sem") is NOT a follow-up."""
    return bool(_FOLLOWUP_CUES.search(question)) or not _clean_fallback_topic(question)
```

question_index.py

```
# src/question_index.py
"""
Structured Question Bank and Vector Index for University Exam Papers.

During document ingestion or rebuild:
1. Every question paper is parsed page-by-page.
2. Individual examination questions are extracted deterministically along with
   metadata: subject, year, marks, page number, question number, semester, university, and source
PDF.
3. Structured QuestionRecords are stored in `_question_bank.json`.
4. A dedicated FAISS vector index (`_question_bank_faiss`) is built using question embeddings,
   enabling semantic, subtopic, and hybrid question retrieval alongside exact matching.
"""

import json
import re
from pathlib import Path
from dataclasses import dataclass, asdict, field
from typing import List, Optional, Dict, Any, Tuple

from langchain_core.documents import Document
from langchain_community.vectorstores import FAISS
from src.document_loader import extract_questions_from_text
```

```
from config import UNIVERSITY_NAME


@dataclass
class QuestionRecord:
    subject: str
    question: str
    year: Optional[str]
    marks: int
    page: int
    source: str
    semester: Optional[str] = None
    university: Optional[str] = UNIVERSITY_NAME
    question_number: Optional[str] = None
    subtopics: List[str] = field(default_factory=list)

    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)

    @classmethod
    def from_dict(cls, data: Dict[str, Any]) -> "QuestionRecord":
        return cls(
            subject=data.get("subject", "UNKNOWN"),
            question=data.get("question", ""),
            year=data.get("year"),
            marks=int(data.get("marks", 0) or 0),
            page=int(data.get("page", 0) or 0),
            source=data.get("source", "unknown"),
            semester=data.get("semester"),
            university=data.get("university", UNIVERSITY_NAME),
            question_number=data.get("question_number"),
            subtopics=data.get("subtopics") or [],
        )


_LAB_RE = re.compile(r'\blabs?\b|\blaborator(?:y|ies)\b|\bpracticals?\b', re.IGNORECASE)


def build_question_bank(question_paper_page_docs, university: Optional[str] = None) ->
List[QuestionRecord]:
    """Extracts structured questions from whole-page question paper Documents.
    Groups pages by source PDF to accurately preserve page numbers and multi-page questions.
    """
    by_source: Dict[str, List[Any]] = {}
    for doc in question_paper_page_docs:
        src = doc.metadata.get("source", "unknown")
        by_source.setdefault(src, []).append(doc)

    records: List[QuestionRecord] = []
    effective_university = university or UNIVERSITY_NAME

    for source, page_docs in by_source.items():
        subject = page_docs[0].metadata.get("subject", "UNKNOWN") if page_docs else "UNKNOWN"
        semester = page_docs[0].metadata.get("semester") if page_docs else None

        # Exclude laboratory practical manuals/assessments from university theory exam bank
        if _LAB_RE.search(source) or _LAB_RE.search(subject):
            continue

        page_texts = [d.page_content for d in page_docs]
        questions = extract_questions_from_text(page_texts)

        for q in questions:
            page_idx = q.get("page", 0)
            year = page_docs[page_idx].metadata.get("year") if 0 <= page_idx < len(page_docs) else
None
            q_num = str(q.get("number", "")).strip() or None
            q_text = str(q.get("text", "")).strip()

            records.append(QuestionRecord(
                subject=subject,
                question=q_text,
                year=str(year) if year else None,
                marks=int(q.get("marks", 0) or 0),
                page=page_idx,
                source=source,
                semester=str(semester) if semester else None,
                university=effective_university,
                question_number=q_num,
                subtopics=[],
            ))
```

```
    return records


def build_question_vectorstore(records: List[QuestionRecord], embeddings, store_path:
Optional[Path] = None) -> Optional[FAISS]:
    """Builds a FAISS vector store embedding each question's text with attached metadata."""
    if not records or embeddings is None:
        return None

    docs = []
    for idx, r in enumerate(records):
        if not r.question or len(r.question.strip()) < 5:
            continue
        metadata = {
            "record_id": idx,
            "subject": r.subject,
            "year": r.year,
            "marks": r.marks,
            "page": r.page,
            "source": r.source,
            "semester": r.semester,
            "university": r.university,
            "question_number": r.question_number,
        }
        docs.append(Document(page_content=r.question, metadata=metadata))

    if not docs:
        return None

    try:
        store = FAISS.from_documents(docs, embeddings)
        if store_path:
            store_path = Path(store_path)
            store_path.mkdir(parents=True, exist_ok=True)
            store.save_local(str(store_path))
        return store
    except Exception as e:
        print(f" Failed to build question vector store: {e}")
        return None


def load_question_vectorstore(store_path: Path, embeddings) -> Optional[FAISS]:
    """Loads existing FAISS vector store for questions if available."""
    p = Path(store_path)
    if not p.exists() or embeddings is None:
        return None
    try:
        return FAISS.load_local(str(p), embeddings, allow_dangerous_deserialization=True)
    except Exception as e:
        print(f" Failed to load question vector store from {store_path}: {e}")
        return None


def save_question_bank(records: List[QuestionRecord], path: Path):
    """Saves QuestionRecord list to JSON."""
    import os as _os
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    # SAFETY_FIX: write to a temp file first; the real file is only replaced once the write is
complete
    tmp = p.with_name(p.name + ".tmp")
    try:
        with open(tmp, "w", encoding="utf-8") as f:
            json.dump([r.to_dict() for r in records], f, indent=2, ensure_ascii=False)
            f.flush()
            _os.fsync(f.fileno())
        _os.replace(tmp, p)
    except BaseException:
        try:
            if tmp.exists():
                tmp.unlink()
        except OSError:
            pass
        raise


def load_question_bank(path: Path) -> List[QuestionRecord]:
    """Loads QuestionRecord list from JSON, returns [] if file does not exist."""
    p = Path(path)
    if not p.exists():
        return []
    try:
```

```
        with open(p, "r", encoding="utf-8") as f:
            raw = json.load(f)
        return [QuestionRecord.from_dict(r) for r in raw]
    except Exception as e:
        print(f" Failed to load question bank ({e}) — treating as empty until next rebuild")
        return []


def search_question_bank(records: List[QuestionRecord], topic: str, subject: Optional[str] = None)
-> Tuple[List[QuestionRecord], int]:
    """Deterministic substring search over question text (backward compatible).
    Returns (matching_records, distinct_paper_count).
    """
    if not topic:
        return [], 0
    topic_lower = topic.lower().strip()
    subject_upper = subject.strip().upper() if subject and subject != "All / Auto-detect" else None

    matches = []
    for r in records:
        if subject_upper and r.subject.strip().upper() != subject_upper:
            continue
        if topic_lower in r.question.lower():
            matches.append(r)

    distinct_papers = len({r.source for r in matches})
    return matches, distinct_papers
```

question_matcher.py

```
# src/question_matcher.py
"""
Multi-Level Hybrid Question Matcher for University Exam Papers.

Supports:
1. Exact / Near-Exact Question Match:
   High lexical overlap, fuzzy token ratio, verbatim substring matching.
2. Semantic / Topic Match:
   FAISS vector similarity retrieval across embedded past exam questions.
3. Subtopic Match:
   Finds questions targeting specific subtopics (e.g., 'Circular Queue', 'Linear Queue').
4. Metadata Filtering:
   Scoping by subject, target marks (e.g., 10 marks), and year constraints.
5. Distinct Frequency Reporting:
   Accurately separates total matching questions from distinct examination papers and distinct exam
years.
"""

import re
import difflib
from dataclasses import dataclass, field
from typing import List, Dict, Set, Optional, Tuple, Any
from langchain_community.vectorstores import FAISS

from src.question_index import QuestionRecord
from src.query_understanding import StructuredQuery


@dataclass
class QuestionMatch:
    record: QuestionRecord
    match_type: str  # "Exact / Near-Exact", "Semantic Match", "Subtopic Match"
    score: float     # 0.0 to 1.0 (higher = better)
    matched_subtopics: List[str] = field(default_factory=list)

    def to_dict(self) -> Dict[str, Any]:
        return {
            "question": self.record.question,
            "year": self.record.year,
            "marks": self.record.marks,
            "page": self.record.page,
            "source": self.record.source,
            "subject": self.record.subject,
            "semester": self.record.semester,
            "question_number": self.record.question_number,
            "match_type": self.match_type,
            "score": round(self.score, 3),
            "matched_subtopics": self.matched_subtopics,
```

```
        }


@dataclass
class MatchResultSet:
    canonical_topic: str
    total_matching_questions: int
    distinct_papers: int
    distinct_years: List[str]
    matches: List[QuestionMatch]
    by_year: Dict[str, List[QuestionMatch]] = field(default_factory=dict)
    marks_breakdown: Dict[int, int] = field(default_factory=dict)
    subject_scoped: bool = False
    total_sittings: int = 0
    coverage_subject: Optional[str] = None

    def to_dict(self) -> Dict[str, Any]:
        return {
            "canonical_topic": self.canonical_topic,
            "total_matching_questions": self.total_matching_questions,
            "distinct_papers": self.distinct_papers,
            "distinct_years": self.distinct_years,
            "matches": [m.to_dict() for m in self.matches],
            "distinct_paper_count": self.distinct_papers,
            "marks_breakdown": self.marks_breakdown,
            "subject_scoped": self.subject_scoped,
        }


def _token_set(text: str) -> Set[str]:
    return set(re.findall(r'[a-zA-Z0-9]+', text.lower()))


def _token_overlap_score(s1: str, s2: str) -> float:
    t1 = _token_set(s1)
    t2 = _token_set(s2)
    if not t1 or not t2:
        return 0.0
    intersection = t1 & t2
    return len(intersection) / len(t1)


def _topic_in_text(topic: str, text: str) -> bool:
    """True when `topic` occurs in `text` starting at a word boundary (both lower-case).
    Longer topics may continue into a suffix ("tree" -> "trees", "hash" -> "hashing"); topics of
    3 letters or fewer ("os", "ai", "ml") must be a whole word, so "os" no longer matches "most"
    and "ai" no longer matches "explain"."""
    if not topic:
        return False
    if len(topic) <= 3:
        pattern = r"(?<![a-z0-9])" + re.escape(topic) + r"s?(?![a-z0-9])"
    else:
        pattern = r"(?<![a-z0-9])" + re.escape(topic)
    return re.search(pattern, text) is not None


class QuestionMatcher:
    """Performs multi-level hybrid retrieval across indexed examination questions."""

    def __init__(self, question_bank: List[QuestionRecord], question_vectorstore: Optional[FAISS] =
None):
        self.question_bank = question_bank
        self.vectorstore = question_vectorstore

    def set_vectorstore(self, store: Optional[FAISS]):
        self.vectorstore = store

    def _match_pool(
        self,
        pool: List[QuestionRecord],
        structured_query: StructuredQuery,
        subject_filter: Optional[str] = None,
        top_n: int = 25
    ) -> Dict[Tuple[str, Optional[str], str], QuestionMatch]:
        matches_dict: Dict[Tuple[str, Optional[str], str], QuestionMatch] = {}

        topic = structured_query.canonical_topic or structured_query.raw_query
        topic_lower = topic.lower().strip()
        subtopics = [st.strip() for st in structured_query.subtopics if st.strip()]

        # ------------------------------------------------------------
        # LEVEL 1: Exact and Near-Exact Lexical Matching
```

```
        # ------------------------------------------------------------
        for r in pool:
            q_text = r.question
            q_lower = q_text.lower()
            key = (r.source, r.year, q_text)

            # Verbatim topic substring match
            if topic_lower and _topic_in_text(topic_lower, q_lower):
                score = 0.95 if topic_lower == q_lower else 0.85
                matches_dict[key] = QuestionMatch(
                    record=r,
                    match_type="Exact / Near-Exact",
                    score=score,
                    matched_subtopics=[topic] if topic else [],
                )
                continue

            # Fuzzy sequence similarity
            ratio = difflib.SequenceMatcher(None, topic_lower, q_lower).ratio()
            if ratio >= 0.75:
                matches_dict[key] = QuestionMatch(
                    record=r,
                    match_type="Exact / Near-Exact",
                    score=ratio,
                    matched_subtopics=[topic],
                )

        # ------------------------------------------------------------
        # LEVEL 2: Subtopic Matching
        # ------------------------------------------------------------
        if subtopics:
            for r in pool:
                key = (r.source, r.year, r.question)
                q_lower = r.question.lower()
                matched_subs = []
                for st in subtopics:
                    st_lower = st.lower()
                    if _topic_in_text(st_lower, q_lower) or difflib.SequenceMatcher(None, st_lower,
q_lower).ratio() > 0.8:
                        matched_subs.append(st)

                if matched_subs:
                    if key in matches_dict:
                        # Upgrade existing match with matched subtopics
                        matches_dict[key].matched_subtopics.extend([s for s in matched_subs if s
not in matches_dict[key].matched_subtopics])
                        matches_dict[key].score = max(matches_dict[key].score, 0.90)
                    else:
                        matches_dict[key] = QuestionMatch(
                            record=r,
                            match_type="Subtopic Match",
                            score=0.88,
                            matched_subtopics=matched_subs,
                        )

        # ------------------------------------------------------------
        # LEVEL 3: Semantic Vector Matching
        # ------------------------------------------------------------
        if self.vectorstore is not None:
            search_phrases = structured_query.search_queries or [topic]
            for phrase in search_phrases[:3]:
                try:
                    scored_docs = self.vectorstore.similarity_search_with_score(phrase, k=top_n)
                    for doc, score in scored_docs:
                        if score > 0.75:  # CUTOFF_PATCH_APPLIED (calibrated, was 1.15)
                            continue

                        meta = doc.metadata or {}
                        rec_id = meta.get("record_id")
                        r = None
                        if rec_id is not None and 0 <= rec_id < len(self.question_bank):
                            r = self.question_bank[rec_id]
                        else:
                            for cand in pool:
                                if cand.question == doc.page_content:
                                    r = cand
                                    break

                        if r is None:
                            continue
```

```
                        if subject_filter and r.subject.strip().upper() !=
subject_filter.strip().upper():
                            continue

                        key = (r.source, r.year, r.question)
                        norm_score = max(0.5, 1.0 - (score / 2.0))

                        if key in matches_dict:
                            matches_dict[key].score = max(matches_dict[key].score, norm_score)
                        else:
                            matches_dict[key] = QuestionMatch(
                                record=r,
                                match_type="Semantic Match",
                                score=norm_score,
                                matched_subtopics=[],
                            )
                except Exception as e:
                    print(f" Vector search error in QuestionMatcher: {e}")
        else:
            search_phrases = structured_query.search_queries or [topic]
            for r in pool:
                key = (r.source, r.year, r.question)
                if key in matches_dict:
                    continue
                q_lower = r.question.lower()
                for phrase in search_phrases:
                    overlap = _token_overlap_score(phrase, q_lower)
                    if overlap >= 0.70:
                        matches_dict[key] = QuestionMatch(
                            record=r,
                            match_type="Semantic Match",
                            score=overlap * 0.85,
                            matched_subtopics=[],
                        )
                        break

        return matches_dict

    def match(self, structured_query: StructuredQuery, top_n: int = 25,
              allow_cross_subject_fallback: bool = True) -> MatchResultSet:
        """Executes multi-level hybrid matching for a StructuredQuery."""
        if not self.question_bank:
            return MatchResultSet(
                canonical_topic=structured_query.canonical_topic,
                total_matching_questions=0,
                distinct_papers=0,
                distinct_years=[],
                matches=[],
            )

        # 1. Subject pool filtering
        subject_filter = structured_query.subject
        pool = self.question_bank
        subject_scoped = False
        if subject_filter and subject_filter != "All / Auto-detect":
            subj_clean = subject_filter.strip().upper()
            filtered_pool = [r for r in self.question_bank if r.subject.strip().upper() ==
subj_clean]
            if filtered_pool:
                pool = filtered_pool
                subject_scoped = True

        matches_dict = self._match_pool(
            pool, structured_query, subject_filter if subject_scoped else None, top_n=top_n
        )

        # Cross-subject fallback: if subject filter was applied but yielded 0 matches,
        # fall back to search across all subjects in the question bank.
        # TREND_FIX: the Exam Intelligence trend passes allow_cross_subject_fallback=False so a
subject
        # with no matches shows "no matches" instead of silently reporting another subject's
questions.
        if subject_scoped and not matches_dict and allow_cross_subject_fallback:
            fallback_matches = self._match_pool(self.question_bank, structured_query, None,
top_n=top_n)
            if fallback_matches:
                matches_dict = fallback_matches
                subject_scoped = False

        topic = structured_query.canonical_topic or structured_query.raw_query

        # ------------------------------------------------------------
```

```
        # LEVEL 4: Explicit Filter Application (Marks, Years)
        # ------------------------------------------------------------
        all_matches = list(matches_dict.values())

        # If specific marks requested (e.g. 10 marks), prioritize or filter
        if structured_query.marks:
            target_marks = structured_query.marks
            # Boost matches matching target marks
            for m in all_matches:
                if m.record.marks == target_marks:
                    m.score += 0.05

        # If specific year constraint
        if structured_query.year_constraint:
            target_year = str(structured_query.year_constraint).strip()
            if target_year.isdigit():
                # Filter to matching year if available
                year_filtered = [m for m in all_matches if m.record.year == target_year]
                if year_filtered:
                    all_matches = year_filtered

        # Sort matches by score descending, then by year descending
        all_matches.sort(
            key=lambda m: (m.score, int(m.record.year) if m.record.year and m.record.year.isdigit()
else 0),
            reverse=True
        )

        # Headline statistics (how many questions, which sittings, marks pattern) count
        # ONLY literal matches: "Exact / Near-Exact" and "Subtopic Match". A "Semantic Match"
        # is just a similar-sounding question (e.g. "Define data structures" for a query on
        # "queue"), so it is still shown to the student as a related question but is never
        # counted as an appearance of the topic. Counted BEFORE the top_n cut so related
        # questions can never push a real match out of the totals.
        counted = [m for m in all_matches if m.match_type != "Semantic Match"]

        # Keep EVERY literal match (needed for exact per-year counts); only the loosely related
        # semantic extras are capped at top_n.
        semantic_extras = [m for m in all_matches if m.match_type == "Semantic Match"][:top_n]
        all_matches = sorted(
            counted + semantic_extras,
            key=lambda m: (m.score, int(m.record.year) if m.record.year and m.record.year.isdigit()
else 0),
            reverse=True,
        )

        # Calculate metrics
        distinct_papers = len({m.record.source for m in counted if m.record.source})
        years_set = {m.record.year for m in counted if m.record.year}
        distinct_years = sorted(list(years_set), reverse=True)

        by_year: Dict[str, List[QuestionMatch]] = {}
        marks_breakdown: Dict[int, int] = {}

        for m in all_matches:
            y = m.record.year or "Unknown Year"
            by_year.setdefault(y, []).append(m)
        for m in counted:
            if m.record.marks > 0:
                marks_breakdown[m.record.marks] = marks_breakdown.get(m.record.marks, 0) + 1

        # Sittings on record for the subject(s) these matches belong to (for "asked in N of M
sittings").
        _subjects = {m.record.subject for m in counted if m.record.subject}
        total_sittings, coverage_subject = 0, None
        if len(_subjects) == 1:
            coverage_subject = next(iter(_subjects))
            total_sittings = len({r.year for r in self.question_bank if r.year and r.subject ==
coverage_subject})

        return MatchResultSet(
            canonical_topic=topic,
            total_matching_questions=len(counted),
            distinct_papers=distinct_papers,
            distinct_years=distinct_years,
            matches=all_matches,
            by_year=by_year,
            marks_breakdown=marks_breakdown,
            subject_scoped=subject_scoped,
            total_sittings=total_sittings,
            coverage_subject=coverage_subject,
        )
```

exam_intelligence.py

```
# CURRICULUM_PATCH_APPLIED
# src/exam_intelligence.py
"""
Exam Intelligence & Student Study Guidance Module.

Analyzes question paper patterns, frequencies, year distributions, and syllabus
alignment to provide fact-grounded study guidance without hallucination.

Provides:
- Distinction between total matching questions and distinct examination papers
- Topic exam history formatting
- Grounded revision guidance separating factual evidence from recommendations
- Repeated questions / top topics identification across the question bank
- Marks-specific question analysis (e.g., 10-mark questions)
- Syllabus cross-referencing
"""

import re
from typing import List, Dict, Optional, Tuple, Any
from pathlib import Path

from src.question_index import QuestionRecord
from src.question_matcher import MatchResultSet, QuestionMatch
from config import SYLLABUS_DIR


_SCAN_NOISE = re.compile(
    r"\b(?:P\.?\s?T\.?\s?O\.?|F\.?\s?T\.?\s?O\.?)\s*\.?|Scanned\s+with\s+\w+(?:\s+Scanner)?|\bGE\
b(?=\s+Scanned)",
    re.IGNORECASE,
)


# Questions that point at a figure/diagram the paper text does not contain
_DIAGRAM_WORDS = (r"(?:graph(?!\s+(?:traversals?|algorithms?|problems?|types|terminolog\w+))|
figure|diagram|"
                  r"tree(?!\s+(?:traversals?|types|operations?|algorithms?|structures?))|circuit|
chart|dfa|nfa|"
                  r"automaton|flowchart|game\s+tree)")
_MISSING_DIAGRAM_RES = [
    re.compile(r"\b(?:following|below|given|shown|above)\s+(?:\w+\s+){0,2}?" + _DIAGRAM_WORDS + r"\
b", re.IGNORECASE),
    re.compile(r"\b" + _DIAGRAM_WORDS + r"\s+(?:given\s+)?(?:below|above|shown)\b", re.IGNORECASE),
    re.compile(r"\bfor\s+(?:the\s+)?(?:following|below|given)\s*[.:]", re.IGNORECASE),
    re.compile(r"\bfig(?:ure|\.)?\s*\d", re.IGNORECASE),
]


def refers_to_missing_diagram(text: str) -> bool:
    """True when a question depends on a figure/graph/tree that is not part of its extracted
text."""
    return bool(text) and any(p.search(text) for p in _MISSING_DIAGRAM_RES)


def clean_question_display(text: str, topic: Optional[str] = None) -> str:
    """Display-only cleanup: removes scanner watermarks, page-turn markers,
    section instructions, and isolates the relevant question clause when multiple
    questions were merged due to OCR noise."""
    cleaned = _SCAN_NOISE.sub(" ", text or "")
    cleaned = re.sub(r'SECTION\s*[-\u2013\u2014:]?\s*[A-Z]\s*(?:Answer\s+any\s+\w+\
s+questions)?.*', '', cleaned, flags=re.IGNORECASE)
    cleaned = re.sub(r'\s+', ' ', cleaned).strip()

    # OCR sometimes glues the NEXT question onto this one after a stray marks digit:
    # "...stack and queues. 4 Define Binary Search Tree. Construct...". Split on
    # sentence-end + 1-2 digit number + capitalised word, and keep only the part(s)
    # that mention the topic (or the first part when none does). Display only —
    # the stored question and all counting stay untouched.
    segs = re.split(r'(?<=[.?!])\s+\d{1,2}\s+(?=[A-Z][a-z])', cleaned)
    if len(segs) > 1:
        keep = [s for s in segs if topic and topic.lower().strip() in s.lower()] or segs[:1]
        cleaned = " ".join(s.strip() for s in keep)

    if topic and len(cleaned) > 160:
        topic_lower = topic.lower().strip()
```

```
        sentences = re.split(r'(?<=[.!?])\s+|(?<=[a-z]\))\s+|(?<=\d\.)\s+', cleaned)
        matching = [s.strip(' ._-') for s in sentences if topic_lower in s.lower() and
len(s.strip()) > 10]
        if matching:
            return ". ".join(matching) + "."

    return cleaned


class ExamIntelligence:
    """Analyzes examination patterns and formulates evidence-based study guidance."""

    def __init__(self, syllabus_dir: Optional[Path] = None):
        self.syllabus_dir = syllabus_dir or SYLLABUS_DIR
        self._syllabus_texts: Optional[Dict[str, str]] = None

    def _ensure_syllabus_loaded(self):
        """Loads text from syllabus PDFs lazily on demand, cached in memory.

        Falls back to OCR per page, same as the main notes/papers loader —
        many of this university's PDFs (including, it turns out, syllabus
        PDFs) are scanned images with no real text layer. Without this, every
        page silently extracts as empty text, self._syllabus_texts ends up
        with entries but no real content, and syllabus_scope() can never
        distinguish "not in the syllabus" from "couldn't read the syllabus" —
        which is exactly why the out-of-syllabus gate never fired on
        anything: it correctly refused to guess when the text was empty.
        """
        if self._syllabus_texts is not None:
            return
        self._syllabus_texts = {}
        if not self.syllabus_dir.exists():
            return

        cache_path = self.syllabus_dir.parent.parent / "faiss_store" / "_syllabus_cache.json"
        if cache_path.exists():
            try:
                import json
                with open(cache_path, "r", encoding="utf-8") as f:
                    self._syllabus_texts = json.load(f)
                if self._syllabus_texts:
                    return
            except Exception:
                pass

        try:
            from pypdf import PdfReader
            from src.document_loader import ocr_page, MIN_TEXT_LENGTH
            for pdf in self.syllabus_dir.glob("**/*.pdf"):
                try:
                    reader = PdfReader(str(pdf))
                    n_pages = len(reader.pages)
                    text_parts = []
                    for i, p in enumerate(reader.pages):
                        if n_pages > 5 and i % 10 == 0:
                            print(f"    Syllabus OCR: page {i + 1}/{n_pages} of {pdf.name}...")
                        txt = (p.extract_text() or "").strip()
                        if len(txt) < MIN_TEXT_LENGTH:
                            ocr_text = ocr_page(pdf, i, dpi=150)
                            if len(ocr_text) > len(txt):
                                txt = ocr_text
                        if txt:
                            text_parts.append(txt)
                    self._syllabus_texts[pdf.name] = "\n".join(text_parts)
                    print(f"    Syllabus OCR done: {pdf.name} ({n_pages} pages)")
                except Exception:
                    pass

            if self._syllabus_texts:
                try:
                    import json
                    cache_path.parent.mkdir(parents=True, exist_ok=True)
                    with open(cache_path, "w", encoding="utf-8") as f:
                        json.dump(self._syllabus_texts, f)
                except Exception:
                    pass
        except Exception as e:
            print(f" Syllabus loading skipped: {e}")

    def syllabus_scope(self, topic: str) -> Optional[bool]:
        """Is this topic inside the syllabus? True / False, or None when that cannot
        be judged (no syllabus uploaded, or its PDFs have no readable text layer —
```

```
        e.g. scans, which this loader does not OCR). None must be treated as
        "don't know" — never as "out of syllabus".

        Deliberately looser than check_syllabus_relevance: a strict majority of the
        topic's significant words (5-letter prefixes, so "Quicksort"/"Quick Sort"
        and plurals still match) must appear anywhere in the syllabus text."""
        self._ensure_syllabus_loaded()
        if not self._syllabus_texts:
            return None
        blob = " ".join(self._syllabus_texts.values()).lower()
        if len(blob) < 3000:   # too little text to trust an absence
            return None
        stop = {"the", "and", "for", "with", "from", "into", "using", "what", "how", "are", "its",
"vs", "versus"}
        tokens = [w for w in re.findall(r"[a-z]{3,}", (topic or "").lower()) if w not in stop]
        if not tokens:
            return None
        keys = [w if len(w) < 5 else w[:5] for w in tokens]
        hits = sum(1 for k in keys if k in blob)
        return hits * 2 > len(keys)

    def check_syllabus_relevance(self, topic: str) -> Optional[Dict[str, Any]]:
        """Checks if a canonical topic is explicitly mentioned in the university syllabus."""
        if not topic:
            return None
        self._ensure_syllabus_loaded()
        if not self._syllabus_texts:
            return None

        topic_clean = topic.lower().strip()
        matches = []
        for pdf_name, text in self._syllabus_texts.items():
            if topic_clean in text.lower():
                # Extract surrounding context (e.g. unit/module name)
                pattern = rf'(.{{0,80}}{re.escape(topic_clean)}.{{0,120}})'
                found = re.findall(pattern, text, re.IGNORECASE)
                snippets = [re.sub(r'\s+', ' ', s).strip() for s in found[:2]]
                matches.append({
                    "syllabus_file": pdf_name,
                    "snippets": snippets,
                })

        if matches:
            return {
                "in_syllabus": True,
                "matches": matches,
            }
        return {"in_syllabus": False, "matches": []}

    @staticmethod
    def format_exam_history_section(match_result: MatchResultSet, max_display: int = 10) -> str:
        """Formats the EXAM HISTORY section with factual paper and question metrics."""
        topic = match_result.canonical_topic
        total_q = match_result.total_matching_questions
        distinct_p = match_result.distinct_papers
        years = match_result.distinct_years

        if total_q == 0:
            msg = (
                f"###  EXAM HISTORY\n\n"
                f"- **Exam Appearances:** No questions matching **'{topic}'** were found in the
currently indexed past examination papers.\n"
                f"- *(Note: This topic may be tested under broader questions, or is from an
unindexed sitting/syllabus module.)*"
            )
            return msg

        years_str = ", ".join(years) if years else "Years not detected"
        # Use the SAME distinct-year count as generate_study_guidance (below)
        # for "sittings" language — these two sections used to compute this
        # number two different ways (this one additionally counted each
        # undated/"Year N/A" record as its own phantom extra "sitting" via a
        # (source, year-or-'unknown') set), so they could show two different
        # sitting counts for the same topic in the same answer. Only a real
        # detected year counts as a sitting now, in both places.
        distinct_year_count = len(years) if years else 0
        sitting_str = (
            f"Appeared across **{distinct_year_count}** examination sitting(s) / paper(s)"
            if distinct_year_count > distinct_p
            else f"Appeared in **{distinct_p}** distinct examination paper(s)"
        )
        lines = [
```

```
            f"###  EXAM HISTORY",
            f"- **Frequency:** {sitting_str} (**{total_q}** total question(s) found).",
            f"- **Exam Years:** {years_str}",
            f"\n**Previous Exam Questions:**",
        ]

        _tot = getattr(match_result, "total_sittings", 0)
        _csub = getattr(match_result, "coverage_subject", None)
        if _tot and _csub and 0 < distinct_year_count <= _tot:
            lines.insert(3, f"- **Coverage:** asked in **{distinct_year_count}** of **{_tot}**
sittings on record for {_csub} ({round(100 * distinct_year_count / _tot)}%).")

        # Display questions sorted by year descending
        literal_matches = [m for m in match_result.matches if m.match_type != "Semantic Match"]
        literal_matches.sort(
            key=lambda m: (int(m.record.year) if m.record.year and m.record.year.isdigit() else 0,
m.score),
            reverse=True,
        )
        shown = literal_matches[:max_display]
        for idx, m in enumerate(shown, 1):
            r = m.record
            y_str = f"**{r.year}**" if r.year else "Year N/A"
            m_str = f"{r.marks} Marks" if r.marks > 0 else "Marks N/A"
            sub_tag = f" `[{m.match_type}]`" if m.match_type != "Exact / Near-Exact" else ""
            # Some extracted "question" records span more than one actual
            # question glued together by OCR noise on the source scan (a
            # missed number boundary). Cap what's displayed so one bad
            # extraction doesn't dump an unrelated wall of text under this
            # topic — the underlying grouping is a separate, deeper fix.
            q_display = clean_question_display(r.question, topic=topic)
            q_display = q_display if len(q_display) <= 280 else q_display[:280].rstrip() + "…"
            diag_tag = "  *(refers to a diagram not shown here)*" if
refers_to_missing_diagram(r.question) else ""
            lines.append(f"{idx}. [{y_str} — {m_str}] {q_display}{diag_tag}
*({r.source})*{sub_tag}")

        if len(literal_matches) > len(shown):
            lines.append(f"\n*(...plus {len(literal_matches) - len(shown)} more question(s) in the
examination database)*")

        return "\n".join(lines)

    def generate_study_guidance(
        self,
        match_result: MatchResultSet,
        syllabus_info: Optional[Dict[str, Any]] = None
    ) -> str:
        """Produces evidence-based revision and exam preparation guidance grounded in data."""
        topic = match_result.canonical_topic
        distinct_p = match_result.distinct_papers
        total_q = match_result.total_matching_questions
        years = match_result.distinct_years
        # NOTE: question papers merge multiple exam sittings/years into one PDF
        # per subject (see project convention), so distinct_papers (distinct
        # source FILES) is not the same as the number of separate exam
        # sittings the topic actually appeared in — that's the number of
        # distinct years instead. Use distinct_year_count for any "sittings"
        # language and for tiering; keep distinct_p only for "paper(s)" wording.
        distinct_year_count = len(years) if years else 0
        breakdown = match_result.marks_breakdown

        lines = ["###  STUDY & REVISION GUIDANCE"]

        # 1. Priority Tier
        if distinct_year_count >= 3 or total_q >= 4:
            priority = "**HIGH EXAM PRIORITY (Repeated Topic)**"
            years_display = ", ".join(years) if years else "multiple years"
            advice = (
                f"**'{topic}'** has appeared repeatedly across **{distinct_year_count}** separate
exam sittings "
                f"({years_display}). "
                "Examiners frequently test this topic. Ensure you can write full, structured
definitions, "
                "draw neat architectural or functional diagrams, and clearly explain operational
workflows."
            )
        elif distinct_year_count >= 1 or total_q >= 1:
            priority = "**MODERATE EXAM PRIORITY (Core Topic)**"
            sittings_disp = f"across **{distinct_year_count}** examination sitting(s)" if
distinct_year_count > 1 else f"in **{distinct_p}** examination paper(s)"
            advice = (
```

```
                f"**'{topic}'** has appeared {sittings_disp}. "
                "While not appearing in every consecutive year, it is a core syllabus concept. "
                "Focus on understanding fundamental definitions, key differences, and standard
operations."
            )
        else:
            priority = "**SYLLABUS-LED PRIORITY (Not in Past Papers)**"
            advice = (
                f"**'{topic}'** did not match any indexed past questions directly. "
                "This does not imply it can be skipped: verify its weightage in your syllabus unit.
"
                "It may be an emerging concept, a lab-oriented task, or framed under a broader
heading."
            )

        lines.append(f"- **Exam Priority Tier:** {priority}")
        lines.append(f"- **Evidence-Based Recommendation:** {advice}")

        # 2. Marks Distribution Insight
        if breakdown:
            # Exact breakdown of how this topic was actually marked (papers use 4/5/6/10
            # marks, so fixed 2-5 / 8-10 buckets used to ignore most questions).
            parts = [f"{mk} marks × {cnt}" for mk, cnt in sorted(breakdown.items())]
            marks_notes = [f"Asked as {', '.join(parts)}."]
            long_count = sum(cnt for mk, cnt in breakdown.items() if mk >= 8)
            if long_count:
                marks_notes.append(f"Tested as a long-answer question (**8+ marks**) {long_count}
time(s). Prepare detailed explanations.")
            if marks_notes:
                lines.append(f"- **Marks Pattern:** {' '.join(marks_notes)}")

        # 3. Syllabus Grounding
        if syllabus_info and syllabus_info.get("in_syllabus"):
            lines.append("- **Syllabus Status:** Verified present in official university syllabus
documents.")
            snippets = syllabus_info.get("matches", [])
            if snippets and snippets[0].get("snippets"):
                lines.append(f"  - Context: *\"{snippets[0]['snippets'][0]}\"*")

        lines.append("\n> [!NOTE]\n> *Study guidance is computed strictly from your university's
indexed examination papers and syllabus documents. It distinguishes observed factual trends from
preparation advice.*")

        return "\n".join(lines)

    @staticmethod
    def get_most_repeated_topics_from_bank(
        question_bank: List[QuestionRecord],
        subject: Optional[str] = None,
        top_k: int = 10,
        min_sittings: int = 2
    ) -> List[Dict[str, Any]]:
        """Analyzes question bank to identify frequently repeated questions and themes."""
        if not question_bank:
            return []

        pool = question_bank
        if subject and subject != "All / Auto-detect":
            subj_clean = subject.strip().upper()
            scoped_pool = [r for r in question_bank if r.subject.strip().upper() == subj_clean]
            # Same fallback pattern used everywhere else subject-scoping
            # happens in this project (_paper_pool_for_subject,
            # QuestionMatcher.match): the LLM's extracted `subject` field is
            # free text from the question ("Data Structures") and often
            # won't match the stored abbreviation code ("DS") exactly. If
            # scoping produces zero results, that almost always means a
            # mismatched subject string, not an actually-empty subject —
            # so fall back to the full pool instead of silently returning
            # no clusters at all.
            if scoped_pool:
                pool = scoped_pool

        if not pool:
            return []

        # Find questions with high similarity across different papers/years
        clusters: List[Dict[str, Any]] = []

        for r in pool:
            q_clean = re.sub(r'[^a-zA-Z0-9\s]', '', r.question.lower()).strip()
            if len(q_clean) < 15:
                continue
```

```
            # Check if fits existing cluster
            placed = False
            for cl in clusters:
                rep_clean = cl["rep_clean"]
                words1 = set(q_clean.split())
                words2 = set(rep_clean.split())
                jaccard = len(words1 & words2) / max(len(words1 | words2), 1)
                if jaccard >= 0.55:
                    cl["questions"].append(r)
                    cl["papers"].add(r.source)
                    if r.year:
                        cl["years"].add(r.year)
                    placed = True
                    break

            if not placed:
                clusters.append({
                    "representative_question": r.question,
                    "rep_clean": q_clean,
                    "subject": r.subject,
                    "marks": r.marks,
                    "questions": [r],
                    "papers": {r.source},
                    "years": {r.year} if r.year else set(),
                })

        # Filter clusters to only those that appeared in at least min_sittings distinct exam
sittings (years)
        repeated_clusters = [cl for cl in clusters if len(cl["years"]) >= min_sittings]

        # Sort clusters by number of distinct exam sittings (years), then question count
        repeated_clusters.sort(key=lambda c: (len(c["years"]), len(c["questions"])), reverse=True)

        results = []
        for cl in repeated_clusters[:top_k]:
            results.append({
                "representative_question": cl["representative_question"],
                "subject": cl["subject"],
                "marks": cl["marks"],
                "distinct_papers": len(cl["papers"]),
                "total_occurrences": len(cl["questions"]),
                "years": sorted(list(cl["years"]), reverse=True),
                "sample_papers": sorted(list(cl["papers"]))[:3],
            })

        return results

    @staticmethod
    def get_questions_by_marks(question_bank: List[QuestionRecord], marks: int, subject:
Optional[str] = None, limit: int = 15) -> List[QuestionRecord]:
        """Returns questions matching a specific marks value."""
        pool = question_bank
        if subject and subject != "All / Auto-detect":
            pool = [r for r in question_bank if r.subject.strip().upper() ==
subject.strip().upper()]

        matching = [r for r in pool if r.marks == marks]
        matching.sort(key=lambda r: (r.year or "", r.source), reverse=True)
        return matching[:limit]
```

rag_pipeline.py – trend, frequency and answering

```
def _is_substantive(question: str) -> bool:
    """Checks if a student query contains actual text beyond punctuation."""
    stripped = re.sub(r'[^A-Za-z0-9]', '', question)
    return len(stripped) >= 2

    def analyze_topic_trend_by_questions(self, topic: str, subject: Optional[str] = None) ->
Optional[Dict[str, Any]]:
        """Exam Intelligence tab: year -> number of matching questions in the question bank."""
        if not topic or len(topic.strip()) < 3:
            return None

        corrected_topic, spelling_corrected = self._correct_topic_spelling(topic.strip())
        pool, subject_scoped = self._paper_pool_for_subject(subject)

        sq = StructuredQuery(
```

```
            raw_query=corrected_topic, canonical_topic=corrected_topic,
            search_queries=[corrected_topic], subject=subject,
        )
        match_res = self.question_matcher.match(sq, allow_cross_subject_fallback=False)  #
TREND_FIX
        counts, undated = self._questions_per_year(match_res.matches)

        base = {
            "topic": corrected_topic,
            "original_topic": topic,
            "spelling_corrected": spelling_corrected,
            "undated_count": undated,
            "subject_scoped": subject_scoped,
            "candidates_checked": len(pool),
        }
        if not counts:
            return {**base, "trend": "insufficient data", "year_counts": {}}
        return {**base, "trend": self._classify_trend(list(counts.keys()), counts), "year_counts":
counts}

    def _classify_trend(sorted_years: List[str], counts: Dict[str, int]) -> str:
        if len(sorted_years) < 2:
            return "insufficient data (only one year with a match)"
        values = [counts[y] for y in sorted_years]
        mid = len(values) // 2
        first_half = values[:mid] if mid > 0 else values[:1]
        second_half = values[mid:] if mid > 0 else values[1:]
        first_avg = sum(first_half) / len(first_half)
        second_avg = sum(second_half) / len(second_half)
        if second_avg > first_avg + 0.5:
            return "rising"
        elif second_avg < first_avg - 0.5:
            return "falling"
        return "stable"

    def _topic_grounded(topic: str, texts: List[str]) -> bool:
        """True when a strict majority of the topic's significant words appear
        (5-letter prefix match, so plurals/spelling variants still count) in the
        retrieved text. Purely lexical — no LLM involved."""
        stop = {"the", "and", "for", "with", "from", "into", "using", "what", "how", "are", "its",
"vs", "versus"}
        tokens = [w for w in re.findall(r"[a-z]{3,}", topic.lower()) if w not in stop]
        if not tokens:
            return True
        blob = " ".join(texts).lower()
        keys = [w if len(w) < 5 else w[:5] for w in tokens]
        hits = sum(1 for k in keys if k in blob)
        return hits * 2 > len(keys)

    def ask(self, question: str, marks: int = 8, subject: Optional[str] = None, context_topic:
Optional[str] = None, plain_answer_only: bool = False) -> Dict[str, Any]:
        """Non-streaming entry point.

        plain_answer_only=True skips exam-history/trend/syllabus/study-guidance
        sections entirely, returning just the concept explanation or code —
        for callers (the Question Paper tab) answering a SPECIFIC real question
        already in hand, where that extra commentary is noise, not insight.
        """
        if not _is_substantive(question):
            return {
                "answer": "I didn't catch an actual question there — could you type what you'd like
to ask?",
                "sources": [],
                "topic": None,
                "debug_info": {"intent": "invalid_input", "llm_calls": 0, "method": "non-
substantive query"}
            }

        if not plain_answer_only:
            _cur = self._curriculum_reply(question)
            if _cur:
                return _cur

        if not plain_answer_only and self._is_diagram_only_request(question):
            return {
                "answer": "Diagrams aren't available in this version of uniRAG — I can only give
text answers. "
                          "Try asking me to *explain* the same topic in words (for
example, \"explain AVL tree rotations\").",
                "sources": [],
                "topic": None,
```

```
                "debug_info": {"intent": "diagram_request", "llm_calls": 0, "method": "diagram
requests are not supported"}
            }

        # 1. Query Understanding
        sq = self.query_understanding.parse(question, context_topic=context_topic,
subject_context=subject)
        understanding_llm_calls = 1 if getattr(self.query_understanding, "last_parse_used_llm",
True) else 0  # REVIEW_FIXES_APPLIED

        if not plain_answer_only:
            _r = self._topicless_reply(sq)
            if _r:
                return _r

        # Question Paper tab: the student is answering ONE real exam question. Exam-meta
        # intents (trend/frequency/...) are never wanted there — words like "increasing
        # order" or "consistent" inside a question must not turn it into a trend report.
        if plain_answer_only:
            _meta = {"frequency", "trend", "importance", "ranking", "exact_questions", "years",
"similar_questions", "syllabus"}
            sq.intents = [i for i in sq.intents if i not in _meta] or ["definition"]
            sq.needs_question_bank = False
            sq.needs_syllabus = False
            if "lab_program" not in sq.intents:
                sq.needs_notes = True

        # 2. Intent-Aware Retrieval & Evidence Gathering
        evidence = self._execute_intent_aware_retrieval(sq, marks, subject,
skip_syllabus_gate=plain_answer_only)

        # 3. Grounded Response Synthesis
        full_answer, synthesis_llm_calls = self._format_grounded_response(evidence, marks, subject,
plain_answer_only=plain_answer_only)
        if plain_answer_only and full_answer and refers_to_missing_diagram(question):
            full_answer = full_answer.rstrip() + (
                "\n\n> ℹ️ This question refers to a figure/graph that is not part of the paper's
text, "
                "so this answer explains the method only and does not use the actual figure.")
        total_llm_calls = understanding_llm_calls + synthesis_llm_calls

        match_res: Optional[MatchResultSet] = evidence.get("question_matches")
        notes_prep = evidence.get("notes_prep") or {}
        result = {
            "answer": full_answer,
            "sources": evidence.get("sources", []),
            "topic": sq.canonical_topic or context_topic,
            "debug_info": {
                "canonical_topic": sq.canonical_topic,
                "subtopics": sq.subtopics,
                "intents": sq.intents,
                "search_queries": sq.search_queries,
                "question_scope": sq.question_scope,
                "is_followup": sq.is_followup,
                "resolved_reference": sq.resolved_reference,
                "llm_calls": total_llm_calls,
                "matching_questions": match_res.total_matching_questions if match_res else 0,
                "distinct_papers": match_res.distinct_papers if match_res else 0,
                "distinct_years": match_res.distinct_years if match_res else [],
                "best_distance": notes_prep.get("best_distance", notes_prep.get("error",
{}).get("debug_info", {}).get("best_distance")),
                "relevance_threshold": MAX_RELEVANT_DISTANCE,
                "method": "LLM Query Understanding → Multi-Level Hybrid Retrieval → Evidence
Aggregation → Grounded Synthesis",
            }
        }
        self._log_query(question, subject, sq, result)
        return result
```

8. Testing

8.1 Introduction

Software testing is an important phase of the development life cycle. It is performed to identify errors, verify functionality and ensure that the application meets its requirements. For uniRAG, testing has an extra purpose: because a language model is part of the system, testing must also check that the guards around the model behave correctly, that numbers shown to the student come from code, and that the application refuses to answer when it should.

8.2 Objectives of Testing

The objectives are to verify that every module works according to its requirement, that exam counts agree with the years listed, that out-of-syllabus questions are declined, that a topic is not carried over into an unrelated question, that lab code is written in the right language (C by default, Java for the OOP lab), that identical questions give identical answers, that unauthorised access and repeated password guessing are prevented, and that data files survive rebuilds and bad input.

8.3 Types of Testing

Unit Testing: Individual functions are tested without Ollama or scanned PDFs, so the tests run in seconds. They cover topic matching, year detection, question extraction, the exam-history text, marks filtering, protection of the question bank during rebuilds, login lockout and upload-name cleaning (`test_uniRAG.py`).

Integration Testing: Verifies the cooperation of query understanding, retrieval, the question matcher, Ollama and the FAISS indexes.

Functional Testing: Checks each feature: login, study questions, frequency, trend, repeated questions, Question Paper Solver and downloads.

Regression Testing: `eval_questions.py` runs 29 fixed cases (44 checks) through the same path the chat tab uses and reports PASS, FAIL or WARN for each rule. Answers are saved to `eval_report.txt`. After every change the suite is run again so that a fix in one place does not break another.

System Testing: The complete application is tested with the real notes, papers and models on the development laptop.

Security Testing: Checks password storage, lockout after five failures, separation of roles, and the handling of a corrupt student file.

Usability and Performance Testing: The interface was reviewed on screen recordings and screenshots, and the response time per answer was measured.

8.4 Test Environment

Testing was carried out on a Lenovo IdeaPad Slim 3 laptop (Intel Core i5-13420H, 16 GB RAM, no GPU) running Windows 11, with Python 3.12, Streamlit, Ollama (mistral:7b and nomic-embed-text), FAISS and Tesseract OCR. The corpus consisted of the MCA notes and question papers of Bangalore University: 22 indexed subject collections and a question bank of 1,426 questions.

8.5 Testing Procedure

Each test case is executed with valid and, where applicable, invalid input. The actual output is compared with the expected output and any difference is recorded as a defect. Defects found during testing were corrected and the affected tests were repeated. Examples of defects found and fixed are a topic from the previous message being carried over into an unrelated question, a trend that showed another subject's data when the matcher fell back to the whole question bank, and a lab-program answer that was not enclosed in a code block. Known model-quality risks, such as a wrong complexity case for quick sort in one explanation, are reported as warnings rather than hidden.

In the saved evaluation run, exam-fact answers took about 25 seconds and explanations and lab programs took from about one to three and a half minutes on the CPU (Chapter 9); this is the main performance limitation and it grows for the Question Paper Solver, which makes one model call per question.

8.6 Test Cases

Test ID	Module	Test Scenario	Expected Result
TC01	Registration	Submit a valid name, e-mail and password	Account is created; only salt and password hash are stored
TC02	Login	Enter valid credentials	Student is logged in and the four study tabs are shown
TC03	Login validation	Enter a wrong password	An error is displayed and the failure is counted
TC04	Lockout	Enter five wrong passwords	The account is locked for 60 seconds
TC05	Administrator login	Log in with the administrator password	System Setup tab becomes visible
TC06	Role separation	Log in as a student	System Setup tab is not shown
TC07	Switch role	Switch role after a chat	Previous chat and session data are cleared
TC08	Definition	Ask “what is decision tree”	Two-part answer: explanation and plain-language analogy, with sources
TC09	Messy input	Ask “wat is normalisation in dbms”	Topic is resolved to Normalization and answered
TC10	Out of syllabus	Ask “explain the French revolution”	The system declines and says the topic is not in the syllabus or papers
TC11	Out of syllabus	Ask “what is a Fenwick tree”	The system declines
TC12	Frequency	Ask how many times a topic has appeared	Count and years come from the question bank and agree with each other
TC13	Mixed language	Ask “queue kitni baar aaya hai exam me”	Understood as the frequency of Queue

Test ID	Module	Test Scenario	Expected Result
TC14	Follow-up	Ask “what about its types” after a topic	Topic is inherited only because the question refers back
TC15	No carry-over	Ask “how many subjects in 1st sem” after a topic	Answered from the curriculum; previous topic is not used
TC16	Lab program	Ask “write a program for binary search”	Code in C (the default language), inside a code block
TC17	Diagram request	Ask to draw a diagram	Declined; a text explanation is offered
TC18	Empty input	Send “?”	The system asks for a real question
TC19	Topic trend	Select CN and ask for a topic's trend	Counts for 2020–2026 from CN questions only, zero-filled
TC20	Subject required	Open the trend without a subject	The user is asked to select a subject
TC21	Most repeated	Select DBMS and press Find	Questions asked in two or more sittings are listed; solutions PDF downloads
TC22	High weightage	Choose a marks level for a subject	Past questions of that mark value are listed
TC23	Question Paper Solver	Upload a question paper PDF	Questions are extracted, each is answered and a file can be downloaded
TC24	OCR	Load a scanned PDF	Pages with no text layer are read with OCR
TC25	Repeatability	Ask the same question twice	Identical answers are returned
TC26	Rebuild	Press Rebuild everything now	Indexes are rebuilt and the question bank is preserved
TC27	Corrupt file	Damage students.json	A backup is kept and the file is not overwritten

Table 8.6.1 List of test cases performed

8.7 Conclusion

Testing of uniRAG combines fast automated unit tests, a regression suite that exercises the full answer path with the local models, and manual functional tests. The guards that keep the model grounded were checked explicitly, and failures found during testing were corrected and the suite re-run. The remaining limitations (literal counting, possible factual slips by the 7B model inside an explanation, OCR noise and CPU-bound speed) are stated openly in Chapter 11 so that the answers are used as study aids and checked against the source books.

9. Results and Discussion

This chapter reports what uniRAG produced when it was run on the real Bangalore University MCA material: the size and make-up of the question bank, how well question matching works at the chosen cut-off, how long answers take, the quality of the repeated-question solutions, and some worked examples. All figures are taken from the project's own data files and evaluation outputs (`_question_bank.json`, `calibration_results.csv`, `eval_report.txt` and the solutions files), not estimated.

9.1 Experimental Setup

All measurements were made on the development laptop: a Lenovo IdeaPad Slim 3 with an Intel Core i5-13420H processor and 16 GB RAM, without a GPU, running Windows 11. Mistral 7B and nomic-embed-text were served by Ollama, notes were stored in per-subject FAISS indexes (chunk size 400, overlap 50, top 3 chunks), and the answer temperature was 0.1 (0 for query parsing).

9.2 Question Bank

Converting the scanned question papers produced a question bank of 1,426 questions. They come from 18 subjects in the first three semesters and from seven exam years, 2020 to 2026. Each record stores the subject, semester, year, marks and the cleaned question text. Table 9.2.1 and Figure 9.2.1 show the number of questions per exam year.

Exam year	2020	2021	2022	2023	2024	2025	2026
Questions	70	114	276	280	326	274	86

Table 9.2.1 Questions in the question bank per exam year

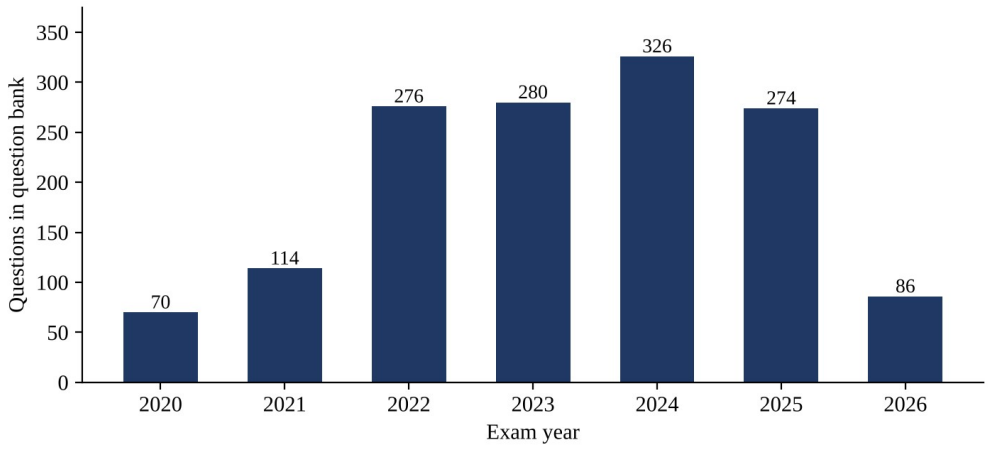


Figure 9.2.1 Questions per exam year

The years 2022 to 2025 carry most of the questions (274 to 326 each), 2021 has 114, and 2020 and 2026 have 70 and 86. The earlier and the latest years have fewer questions in the bank, so a topic's yearly count in those years is based on fewer papers. This is why the trend

chart shows zero-filled years and should be read together with the number of questions available in that year.

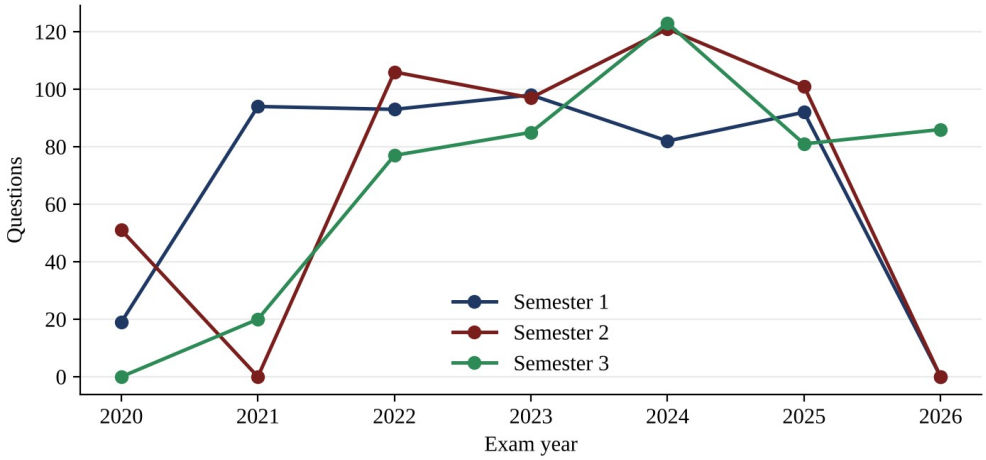


Figure 9.2.2 Questions per exam year in each semester

The line chart shows that every semester has papers from five or six of the seven exam years, and that the gaps differ. Semester 1 has no 2026 papers, Semester 2 has no 2021 and 2026 papers, and Semester 3 has no 2020 papers but is the only semester with 2026 papers (86 questions). A line falls to zero where a semester has no papers for that year. A topic's year-by-year count therefore depends on which papers were available, and a zero in the trend chart may mean that no paper of that year exists, not that the topic was not asked.

Semester	Subject (number of questions)	Total
1	ACP (75), CA (88), DM (113), DS (111), TOC (91)	478
2	AI (70), CN (86), DAA (88), DBMS (75), OS (91), SE (66)	476
3	BDA (78), CC (76), CNS (14), ML (31), QTR (118), RM (73), WP (82)	472

Table 9.2.2 Questions per subject and semester

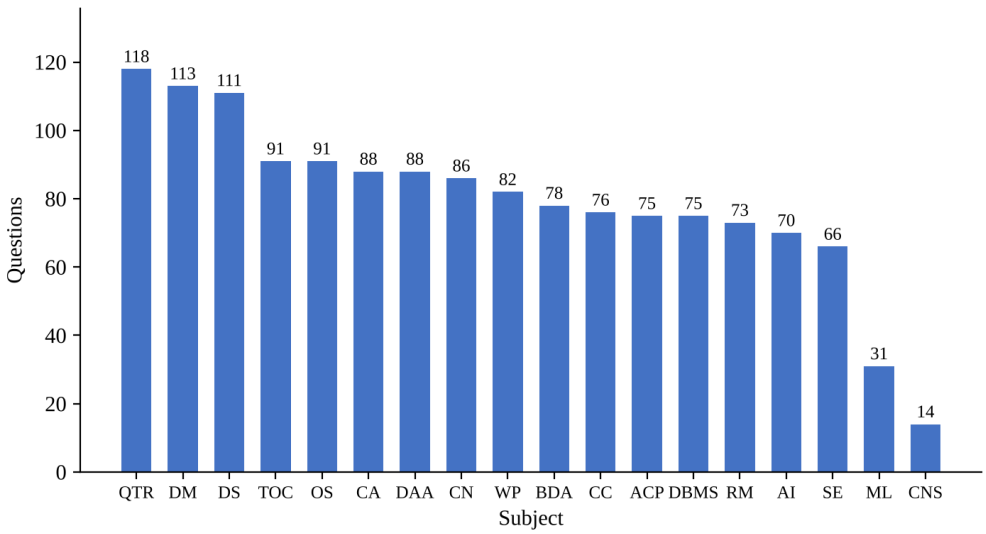


Figure 9.2.3 Questions per subject

Subjects differ widely in size: QTR (118), DM (113) and DS (111) have the most questions, while CNS (14) and ML (31) have the fewest. For the subjects with few questions, counts such as “asked three times” rest on a very small sample, and a missing topic may simply not have been in the available papers.

Marks	10	8	7	6	5	4	3	2
Questions	251	3	7	692	336	59	69	9

Table 9.2.3 Questions by marks allotted

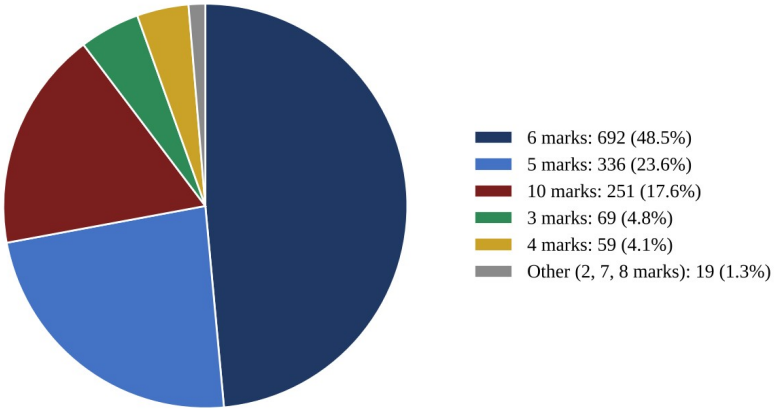


Figure 9.2.4 Share of questions by marks allotted

Questions of 6 marks (692, about 49 per cent) and 5 marks (336) make up most of the bank, and 251 questions carry 10 marks. This distribution is what the high-weightage view and the marks pattern in the study guidance are computed from.

9.3 Calibration of the Question-Matching Cut-off

The semantic level of the question matcher accepts a stored question only if the distance between its embedding and the query embedding is below a cut-off. A cut-off that is too strict misses related questions; one that is too loose shows unrelated ones. To choose it by measurement rather than by guess, a labelled set of 61 test queries was prepared (`calibration_set.csv`): 46 queries for which a correct question exists in a given subject, and 15 queries for which nothing in the papers should match. The script `calibrate_cutoff.py` embeds each query, retrieves questions at each cut-off from 0.30 to 1.20 and compares them with the answer key. It does not call the language model.

Cut-off	Precision	Recall	F1	Queries with a correct hit	Nothing-queries wrongly answered	Wrong questions shown
0.4	1.0	0.135	0.238	15/46	0/15	0
0.5	0.989	0.298	0.457	35/46	0/15	1
0.6	0.907	0.474	0.623	42/46	0/15	14
0.7	0.855	0.651	0.739	44/46	1/15	32
0.75	0.777	0.737	0.757	44/46	1/15	61
0.8	0.678	0.796	0.732	45/46	4/15	109

Cut-off	Precision	Recall	F1	Queries with a correct hit	Nothing-queries wrongly answered	Wrong questions shown
0.9	0.49	0.879	0.629	46/46	7/15	264
1.15	0.285	0.917	0.435	46/46	15/15	665

Table 9.3.1 Matching quality at selected cut-offs

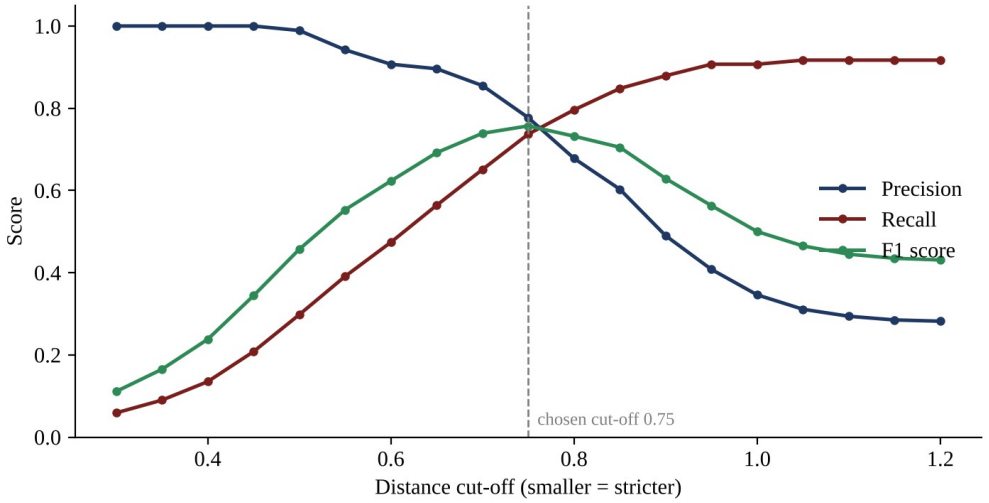


Figure 9.3.1 Precision, recall and F1 score against the cut-off

The results show the expected trade-off. At a strict cut-off of 0.50 precision is almost perfect (0.989), but recall is only 0.298, so most related questions are missed. At the original permissive value of 1.15, recall is high (0.917) but precision falls to 0.285, all 15 nothing-queries are wrongly answered and 665 wrong questions are shown across the test set. The best balance is at 0.75, where the F1 score peaks at 0.757 (precision 0.777, recall 0.737), a correct hit is found for 44 of the 46 queries, and only 1 of the 15 nothing-queries is wrongly answered. The cut-off in `question_matcher.py` was therefore changed from 1.15 to 0.75.

The result should be read with care: the test set is small and was prepared for this project, so it shows that 0.75 is a sound choice for these papers but not that it is optimal for every university. The exact and subtopic levels of the matcher are not affected by this cut-off.

9.4 Regression Suite and Response Time

The regression suite (`eval_questions.py`) runs fixed questions through the same code path as the chat tab and checks explicit rules, such as no topic carry-over, refusal of out-of-syllabus topics, consistent counts and years, and identical answers for identical questions. The saved evaluation run recorded 30 question runs. Table 9.4.1 groups their response times.

Type of question	Runs	Fastest (s)	Median (s)	Slowest (s)
Out-of-syllabus (declined)	3	0	0	0
Exam facts (frequency, exact questions, ranking, trend)	11	23	25	112
Explanation of a topic	12	51	77	214

Type of question	Runs	Fastest (s)	Median (s)	Slowest (s)
Comparison of two topics	2	104	131	157
Lab program	2	173	193	212

Table 9.4.1 Response time by type of question (seconds)

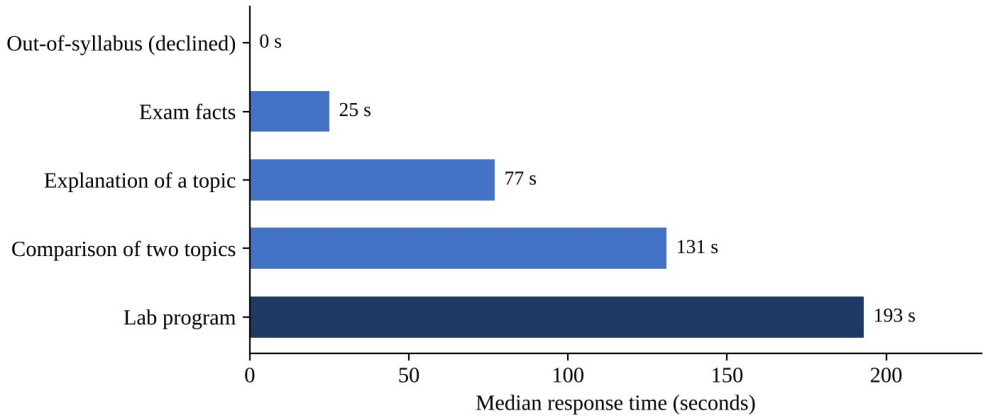


Figure 9.4.1 Median response time by type of question

Questions that are declined are answered instantly, because the syllabus gate stops them before any model is called. Exam-fact questions take about 25 seconds, because the language model is used only to parse the question and the counts are produced by code. Explanations take roughly one to three and a half minutes, and lab programs the longest (about three minutes), because the model must generate long text on a CPU. A saved answer from the answer cache is returned in seconds, which is why frequently asked and demonstration questions were pre-computed with `warm_cache.py`.

9.5 Repeated Questions and Solutions Bundle

The Most Repeated Questions view lists questions that were asked in two or more exam sittings of a subject. Across 16 subjects the question bank contains 125 repeated questions (ML and CNS have none). A solutions PDF bundles one model answer for each. Of the 125 answers, 67 are curated (drafted with an AI assistant, checked, and numerical results recomputed in code) and 58 were generated offline by the local 7B model from the notes. The local model was unreliable on calculation and proof questions: 41 of its first 99 generated answers were wrong or did not solve the question and were replaced by curated ones. The 58 that remain are mostly descriptive theory and were only skimmed, so they should be read with more care than the curated answers.

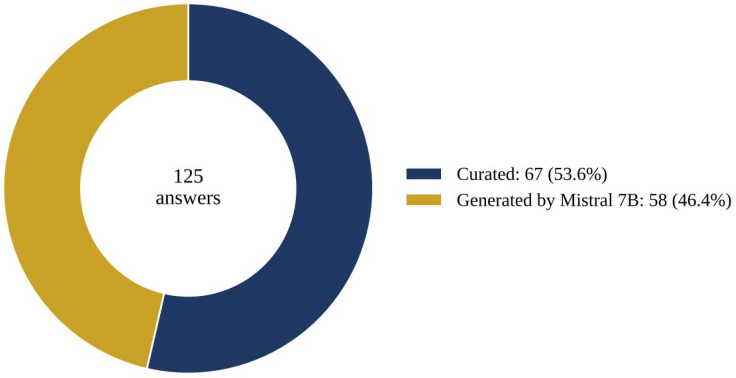


Figure 9.5.1 Source of the 125 repeated-question answers

9.6 Worked Examples

Exam history of a topic

For the question “what is queue? linear and circular. how many times it appeared…” with the subject DS selected, uniRAG reported that the topic appeared in 5 examination sittings (6 questions in total) in the years 2025, 2024, 2023, 2022 and 2020, which is 5 of the 6 sittings on record for DS (83 per cent). It listed the six questions with year and marks, assigned the priority tier “High exam priority (repeated topic)”, and gave the marks pattern 4 marks × 2, 5 marks × 2 and 6 marks × 2. All of these numbers were computed from the question bank and were identical on repeated runs.

Year-by-year trend

For the topic “Queue” the trend view showed 2 matching questions in 2020, none in 2021, and one in each of 2022 to 2025, and classified the trend as stable. Earlier in development the same screen showed the Data Structures chart when Computer Networks (CN) was selected, because the matcher fell back to the whole question bank when the selected subject had no match. The trend now uses only the selected subject, with the cross-subject fallback switched off, so a subject with no match shows an empty result instead of another subject's data.

Declining an out-of-syllabus question

The questions “what is a Fenwick tree”, “explain Kubernetes” and “explain the French revolution” were each declined with the message that the topic does not appear in the MCA syllabus or in any past question paper, without calling the language model. In contrast, “what is health” was answered, because the word occurs in the cryptography notes (health testing of a noise source), and the answer covered both meanings.

9.7 Discussion

What works well. Separating the work of the model from the work of the code gave answers that can be trusted for numbers: counts, years, marks and trends are exact, repeatable and

traceable to the question bank. The guards prevented answers from unsupported text, and the calibrated cut-off removed most unrelated matches. Because everything runs locally, the system is private and has no running cost, and a saved answer is returned in seconds.

Limitations seen in the results. Counting is literal, so a topic worded differently in a paper can be missed; this is the deliberate price of never letting the model invent a count. Small subjects (CNS, ML) and the earliest and latest years have few questions, so their statistics are weaker. The 7B model can still slip on a fact inside an explanation: for example, in testing it once gave the wrong time-complexity case for quick sort. Lab programs and long explanations take several minutes on a CPU. Some scanned questions contain OCR noise or have no detected year and are left out of year-by-year counts. Finally, the calibration set is small, so the chosen cut-off should be re-measured when a new university's papers are added.

Overall. The results show that a laptop-only, offline assistant can give a student useful, grounded explanations and reliable exam statistics for a whole programme, as long as its answers are treated as study aids and key facts are confirmed in the source books that every answer lists.

10. Screenshots

The uniRAG application provides a login screen, a Study Assistant with structured answers, a Question Paper Solver, an Exam Intelligence area, a Resource Library and an administrator Setup area. The following figures show the main screens of the application.

Login page

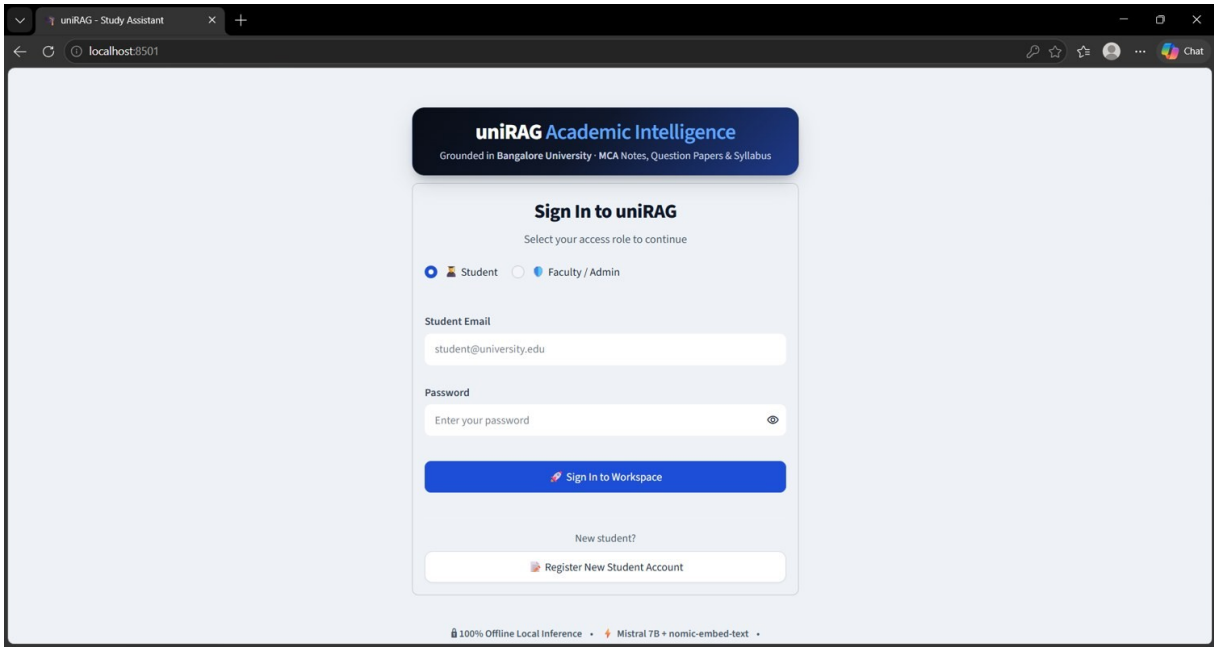


Figure 10.1 Login Page

Create account

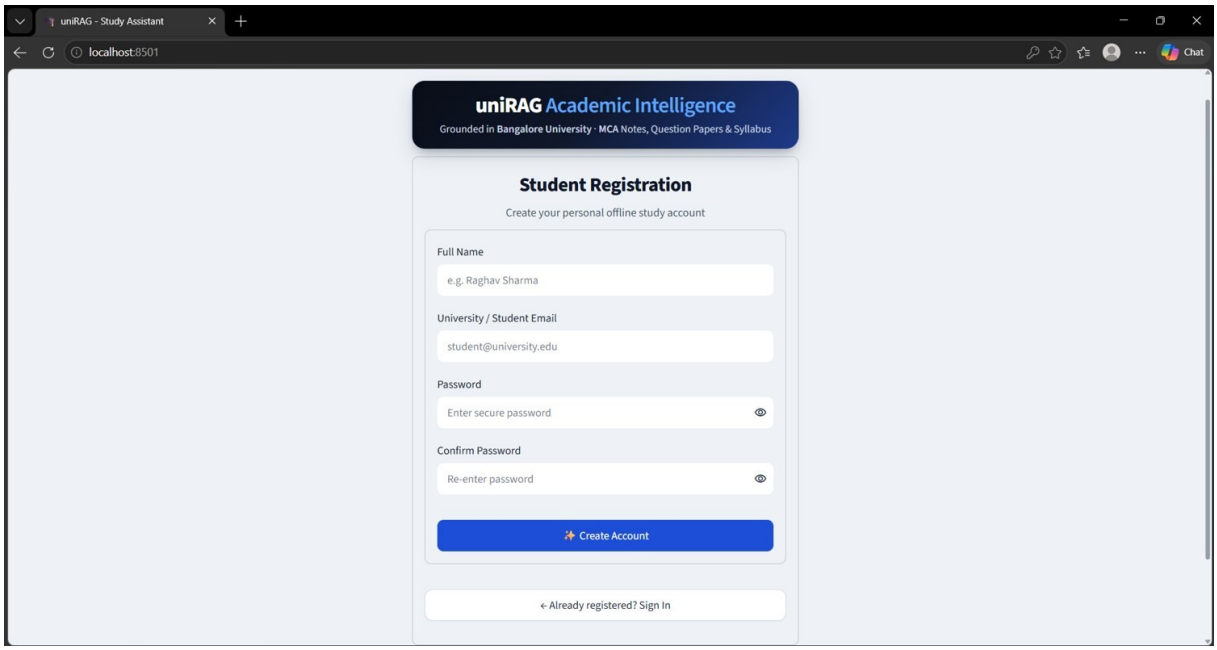


Figure 10.2 Account creation page

Study Assistant home

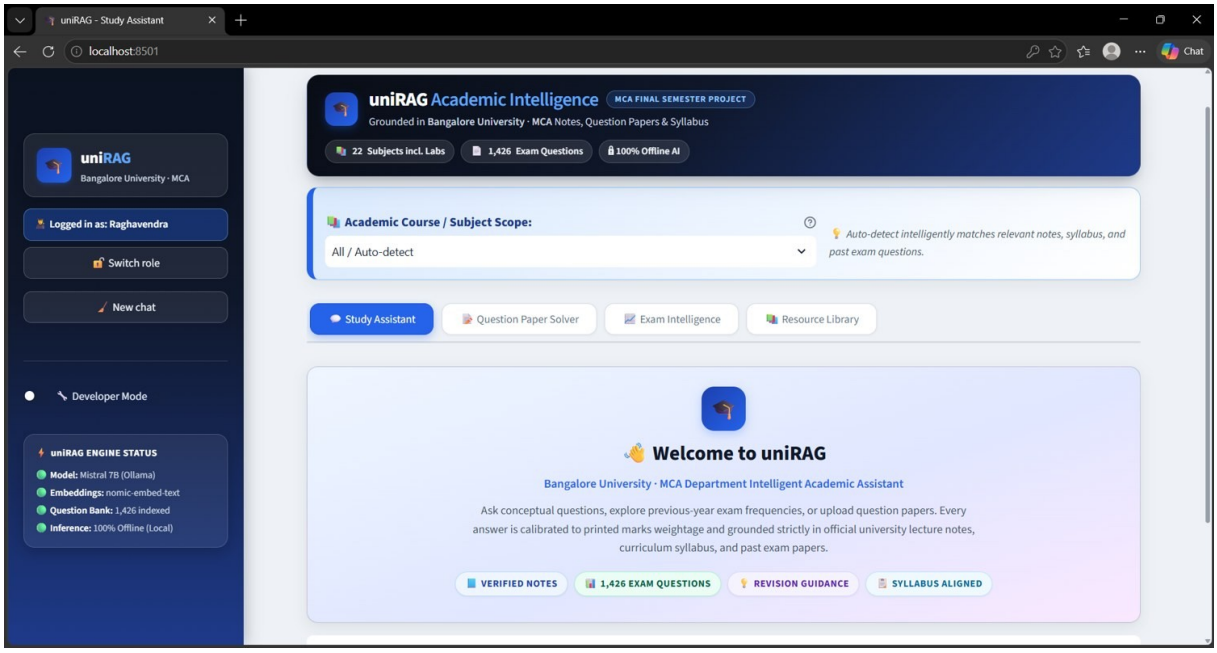


Figure 10.3 Study Assistant home and subject selection

Study Assistant answer

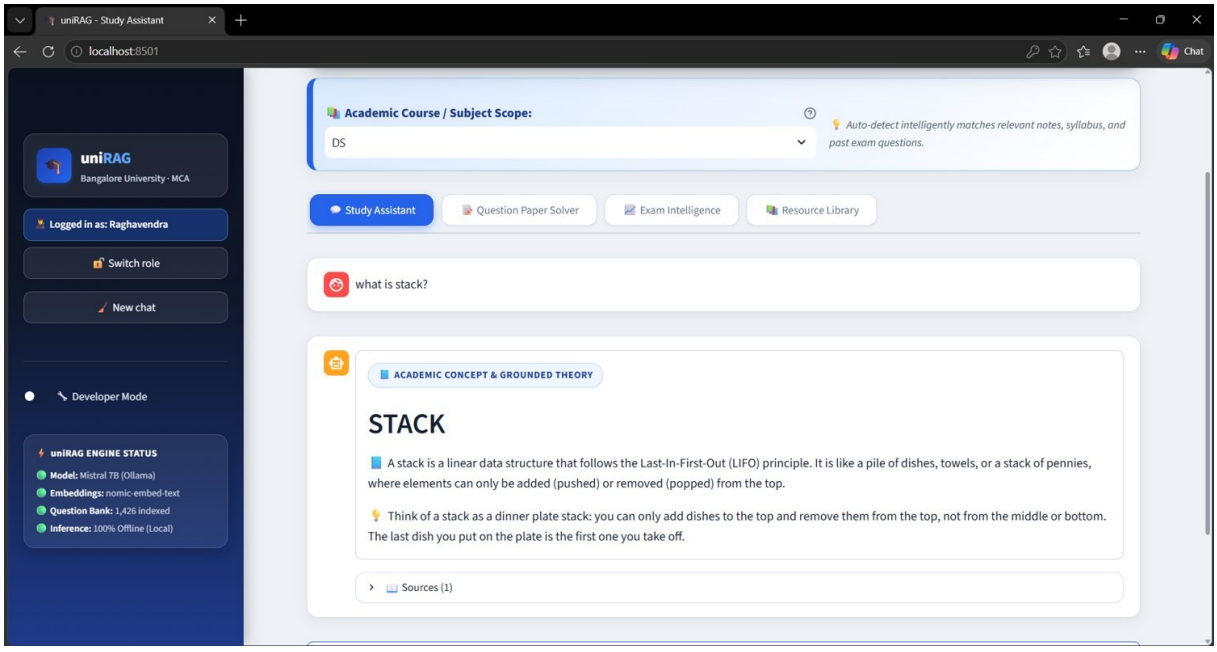


Figure 10.4 Structured answer

Exam history

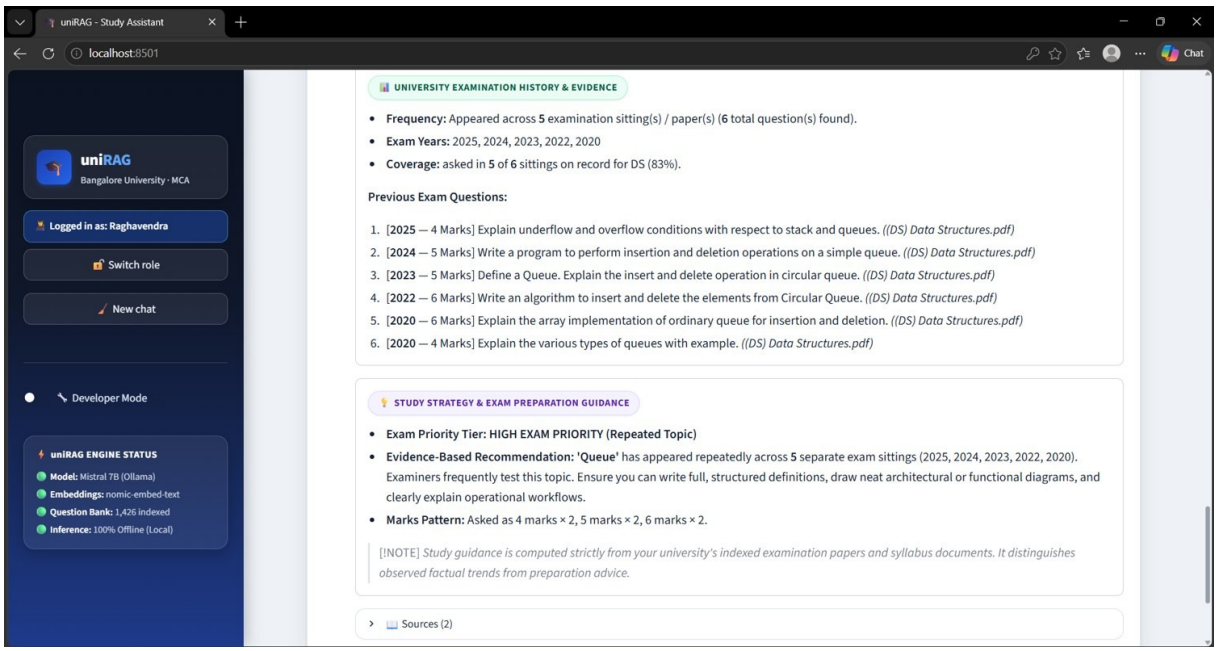


Figure 10.5 Exam history of a topic

Question Paper Solver

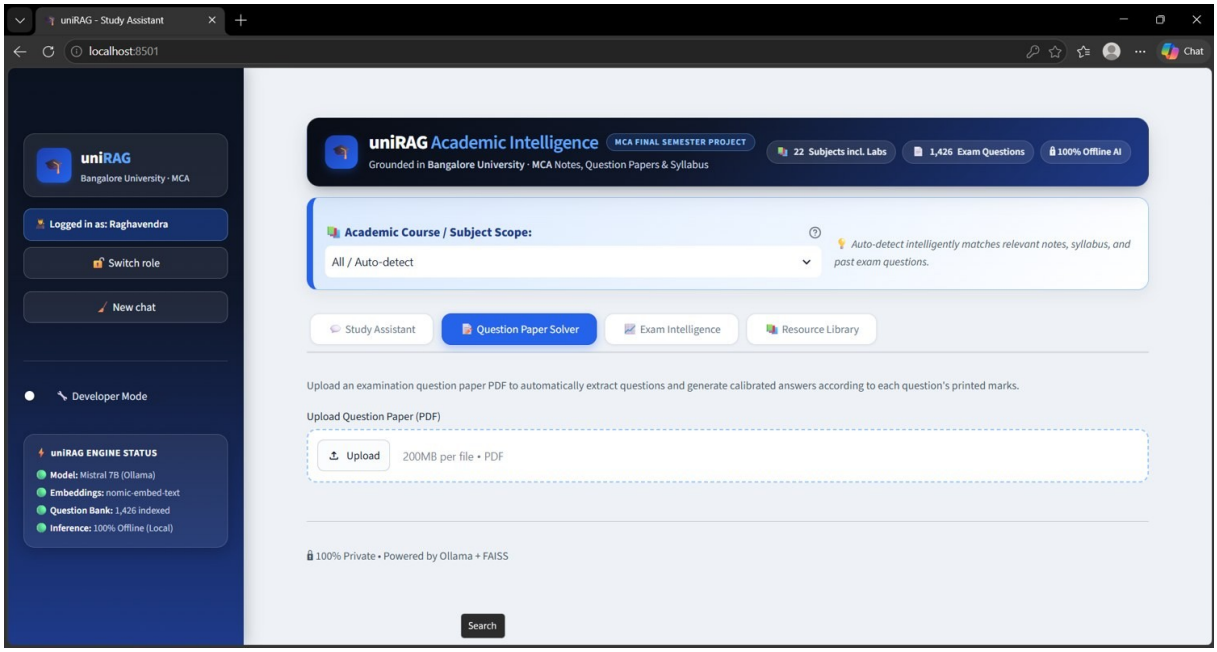


Figure 10.6 Question Paper Solver

Exam Intelligence overview

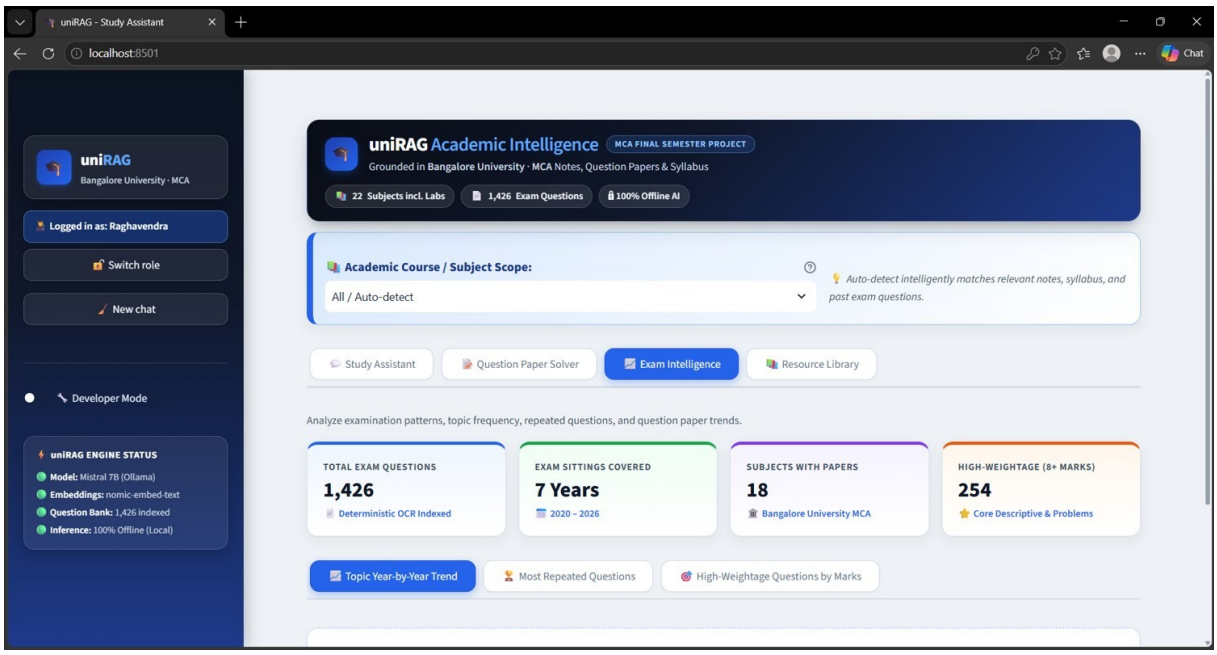


Figure 10.7 Exam Intelligence overview

Topic Year-by-Year Trend

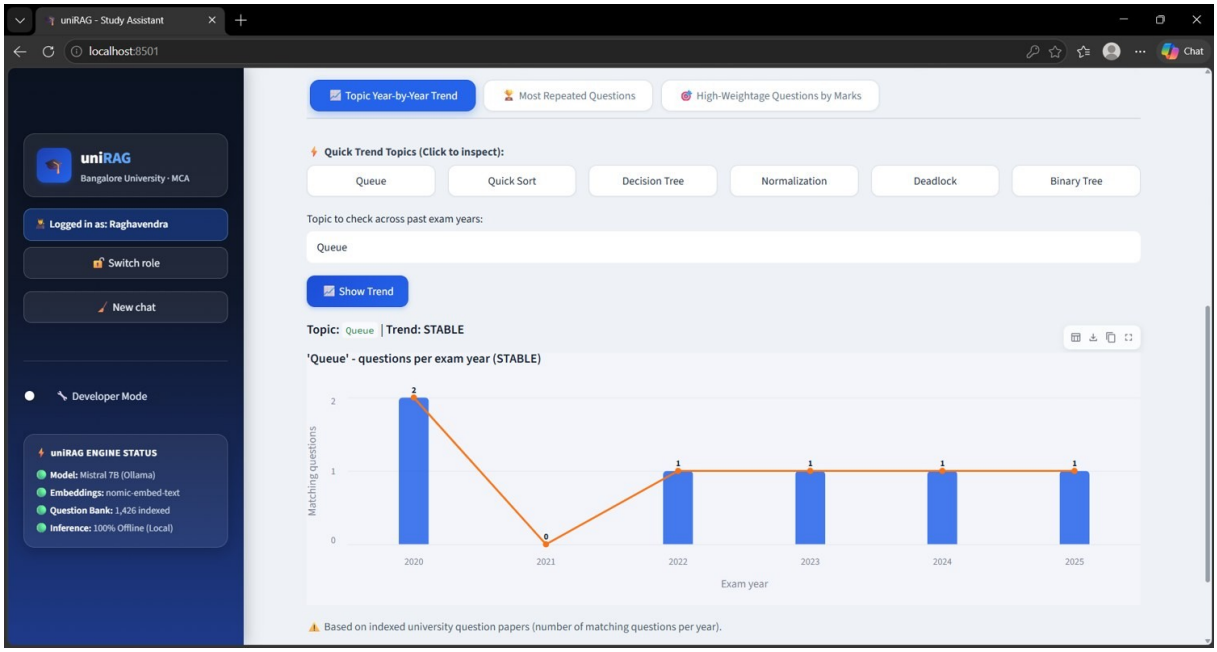


Figure 10.8 Topic year-by-year trend

Most Repeated Questions

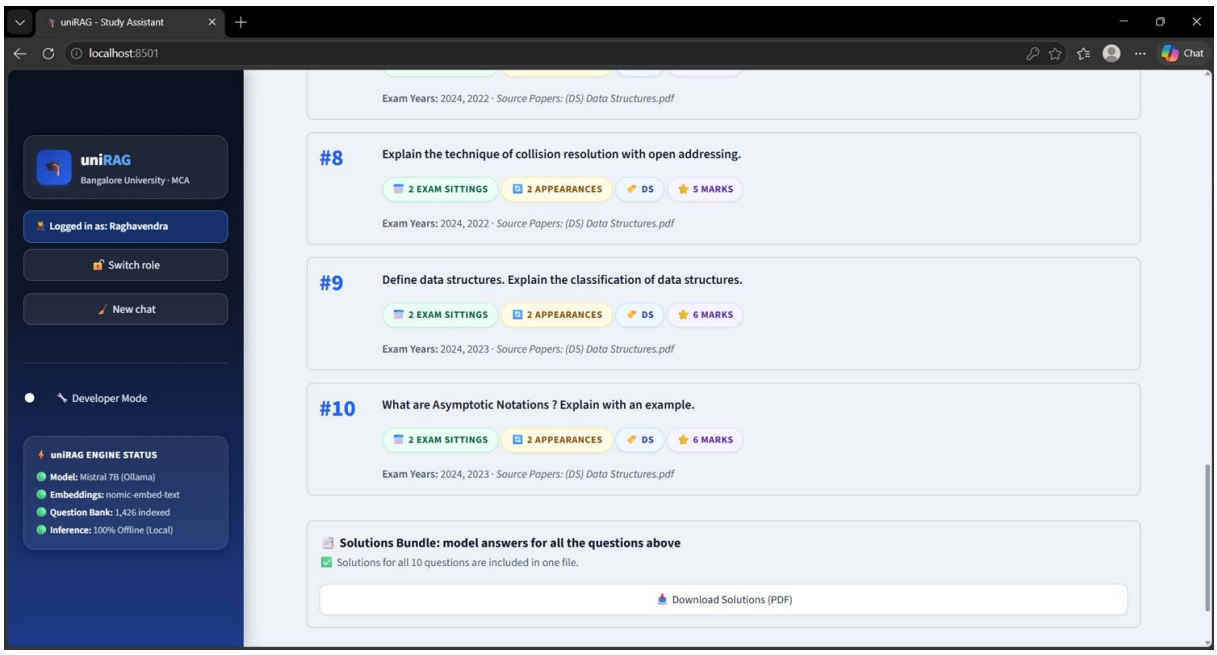


Figure 10.9 Most repeated questions and solutions bundle

High-Weightage Questions by Marks

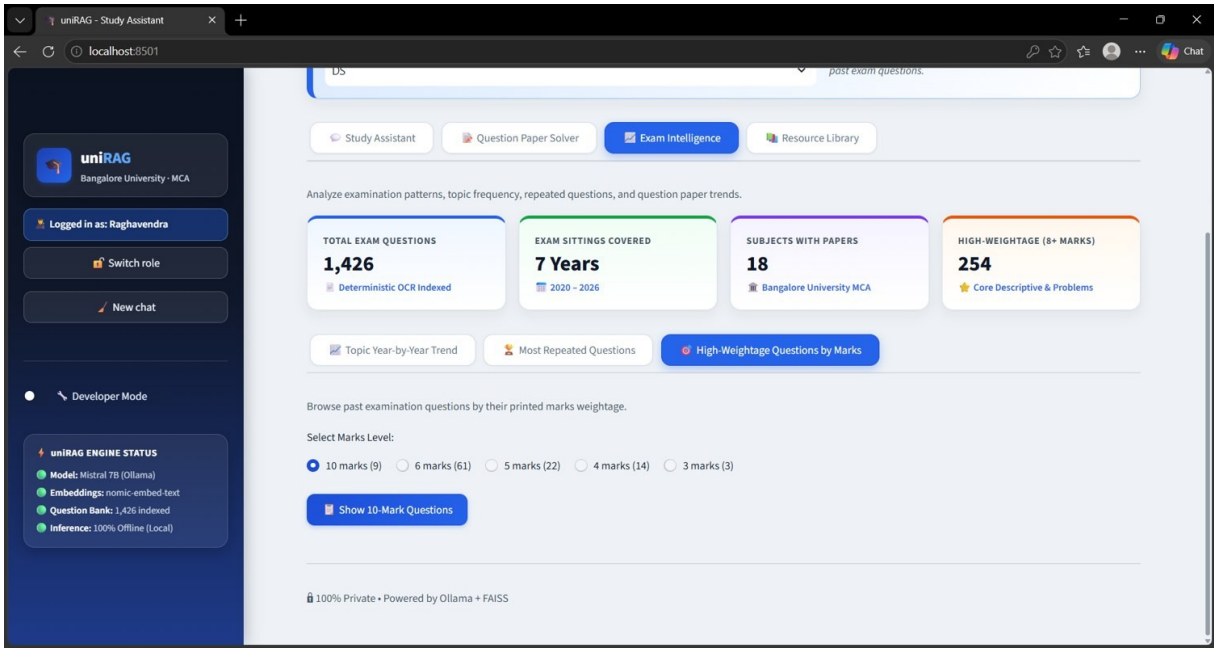


Figure 10.10 High-weightage questions by marks

Resource Library

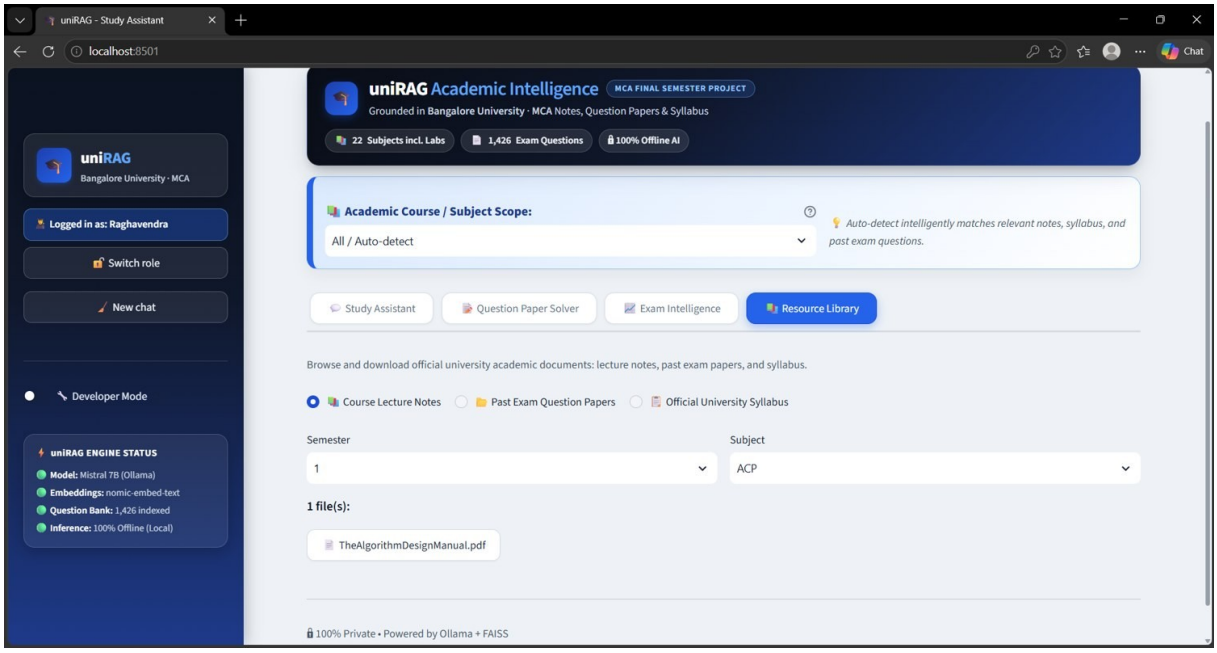


Figure 10.11 Resource Library

System Setup (administrator)

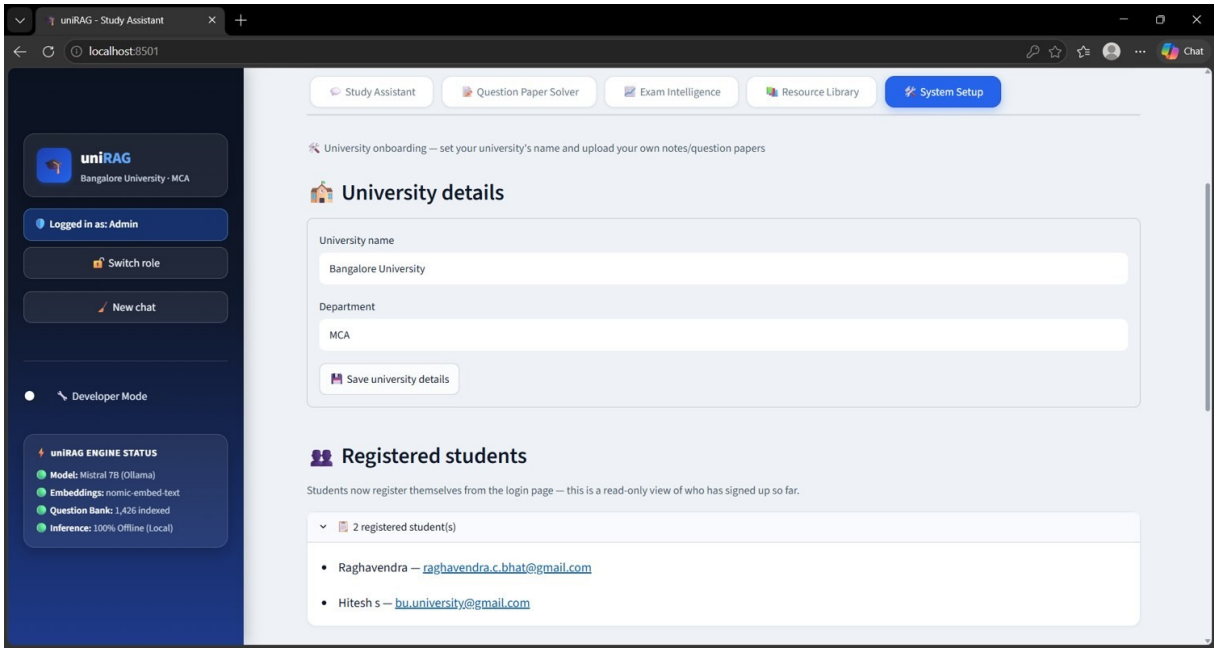


Figure 10.12 System Setup

Upload document and rebuild index

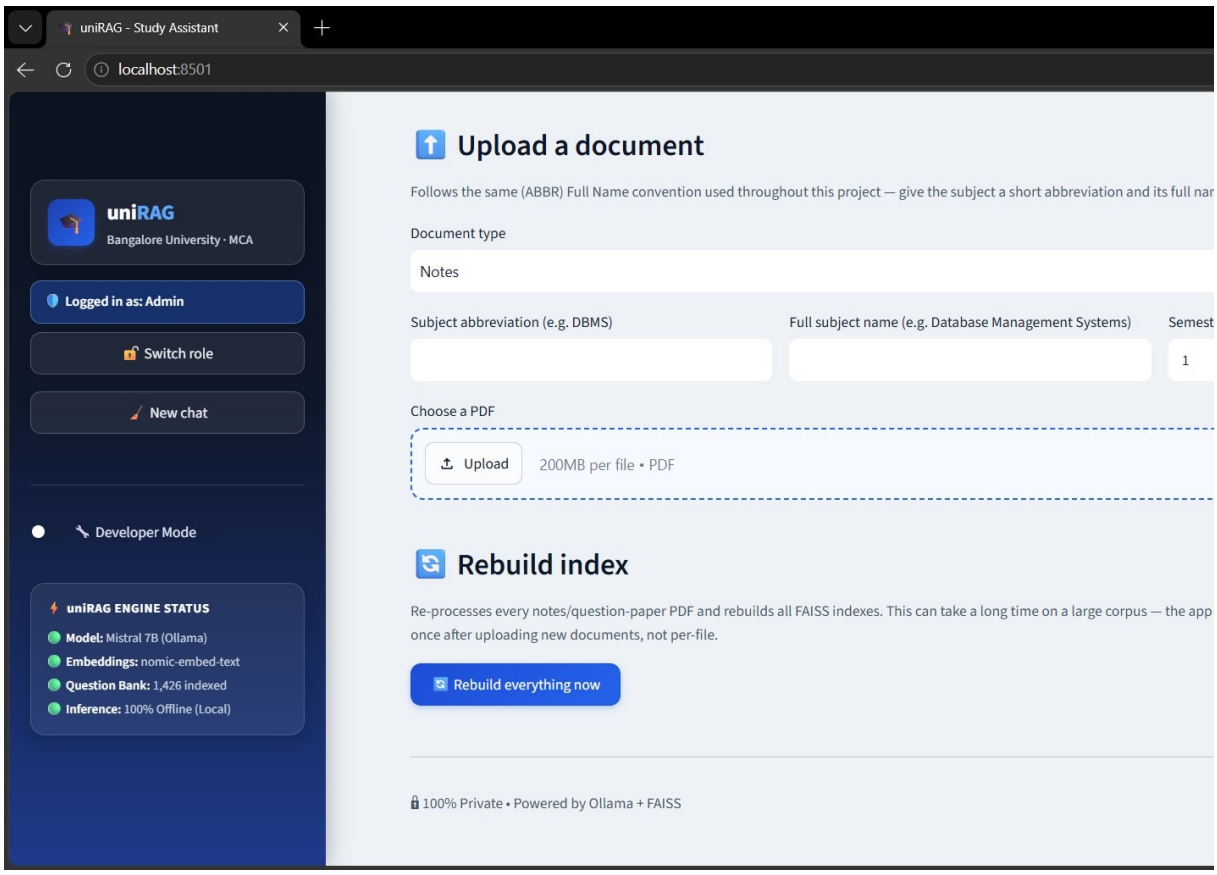


Figure 10.13 Upload a document and rebuild the index

11. Conclusion

uniRAG is an offline, retrieval-augmented study assistant for the MCA programme of Bangalore University. It gives a student explanations drawn from the university's own notes, and facts about the examination history of a topic drawn from a structured question bank of 1,426 questions covering seven exam years. The whole system runs on one CPU-only laptop using Python, Streamlit, Ollama with Mistral 7B and nomic-embed-text, FAISS and Tesseract OCR, so no data leaves the machine and no running cost is involved.

The central idea of the project is that the language model writes, and code counts and checks. All numbers (how many times, which years, how many marks, the trend) are produced by deterministic code, and a set of guards (relevance cut-off, topic-in-context check, out-of-syllabus gate, deterministic counting, follow-up check, figure-reference stripping and empty-input check) prevents the model from answering when the notes do not support it. The system can therefore say “I have not answered this because it is outside your syllabus” instead of producing a confident but unsupported answer.

Beyond the chat, the application provides a Question Paper Solver, an Exam Intelligence module with topic trends, most repeated questions with a solutions PDF and high-weightage questions, a Resource Library and an administrator Setup area. Accounts are protected with salted PBKDF2 hashes and a lockout, and data files are written safely and backed up. A regression suite of 29 cases and automated unit tests were used to keep the behaviour stable while the system was improved.

12.1 Limitations

- **Counting is literal.** A topic worded differently in the papers can be missed. This is deliberate: it is the price of never letting the model invent a count.
- **The model can still state a wrong fact** in an explanation. Guards limit what it is asked to write but cannot verify every sentence, so explanations are study aids and the source books are listed.
- **OCR noise.** Scanned papers occasionally leave stray characters, and some questions have no detected year and are excluded from year-by-year answers.
- **Semantic matching cut-off** was calibrated on a labelled set of 61 queries (Chapter 9), but that set is small and should be re-measured for another university's papers.
- **No diagrams.** Figures are not extracted or drawn.
- **CPU-only speed.** Exam-fact answers take about half a minute, but explanations and lab programs take one to three and a half minutes, which is slow for long batches.
- **Solutions bundle.** Of 125 repeated questions across 16 subjects, 67 have curated answers and 58 were generated offline by the local 7B model and only skimmed.
- **Security scope.** Registration is open with no e-mail verification or password reset; the system is meant for an offline single-machine deployment and not for the internet.

12. Future Enhancements

Although uniRAG provides the useful features described in this report, there are many opportunities to improve its accuracy, speed and reach. The following enhancements can be considered in future versions.

12.1 Calibrated Matching

The question-matching cut-off was calibrated on a labelled set of 61 queries. A larger and more varied labelled set, covering all subjects and written by more than one person, would make the choice more reliable, and the notes-retrieval cut-off (1.15) should be calibrated in the same way. Per-subject cut-offs could also be considered.

12.2 Answer Verification

A small verifier could check each sentence of an explanation against the retrieved text and flag or remove sentences that the notes do not support. This would reduce the risk of the model stating a wrong fact inside an otherwise grounded answer.

12.3 GPU and Faster Inference

Support for a GPU, a quantised or smaller model for the parsing step, and better caching would reduce the response time from minutes to seconds and make the Question Paper Solver practical for long papers.

12.4 Diagram and Table Extraction

At present figures and diagrams are not extracted or drawn, and requests to draw something are declined. Figures, tables and circuit or tree diagrams in notes and papers could be extracted and shown, and questions that refer to a figure could then be answered fully instead of explaining only the method.

12.5 Multilingual Support

Students already mix English with Hindi or Kannada words in their questions. Full support for Kannada and Hindi questions and explanations would make the assistant more accessible.

12.6 Account System and Network Deployment

At present registration is open, there is no e-mail verification and the system is meant for one machine, so internet-facing deployment is not supported. A proper account system with e-mail verification, password reset, per-student history and a server deployment on a campus network would allow many students to use one installation securely.

12.7 Revision and Practice Tools

The question bank could drive practice tests, flash cards and a revision planner that schedules topics by exam weightage and by the student's own progress.

12.8 Additional Programmes and Universities

The folder-based curriculum and the Setup tab already allow another university to be configured. Ready-made packs for other programmes and semesters, and automatic detection of new papers, would extend the system.

12.9 Better Analytics

Optional, privacy-preserving usage statistics (for example which topics are asked most) could help faculty see where students need support, and a feedback button could let students report incorrect answers.

13. References

1. Python Software Foundation, Python Documentation. https://docs.python.org/3/

2. Streamlit Inc., Streamlit Documentation. https://docs.streamlit.io/

3. Ollama, Ollama Documentation. https://docs.ollama.com/

4. A. Q. Jiang et al., Mistral 7B, arXiv:2310.06825, 2023. https://arxiv.org/abs/2310.06825

5. P. Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401

6. J. Johnson, M. Douze and H. Jégou, Billion-scale similarity search with GPUs, arXiv:1702.08734, 2017. https://arxiv.org/abs/1702.08734

7. Facebook AI Research, FAISS Library. https://github.com/facebookresearch/faiss

8. Z. Nussbaum et al., Nomic Embed: Training a Reproducible Long Context Text Embedder, arXiv:2402.01613, 2024. https://arxiv.org/abs/2402.01613

9. LangChain, LangChain Documentation. https://python.langchain.com/docs/

10. Tesseract OCR, Tesseract Open Source OCR Engine. https://github.com/tesseract-ocr/tesseract

11. Artifex Software, PyMuPDF Documentation. https://pymupdf.readthedocs.io/

12. Python Software Foundation, hashlib – Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

13. OWASP, Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

14. Altair Developers, Altair: Declarative Visualization in Python. https://altair-viz.github.io/

15. Ministry of Education, Government of India, All India Survey on Higher Education (AISHE) 2021-22. https://education.gov.in/sites/upload_files/mhrd/files/statistics-new/AISHE%20Book_2021-22_4.pdf